

Quaderno di lavoro

Che cosa è cambiato e perché

Settembre 2026 — dal laboratorio emulato
al datapath reale

Progetto **IPA/eBPF**

inferenza di una rete neurale dentro il kernel, a bordo del pacchetto

Ogni numero di questo quaderno viene da una misura eseguita,
non da una stima. Dove una cosa non è stata verificata, è scritto.

Indice

1	Di che cosa stiamo parlando	4
1.1	L'idea in una pagina	4
1.2	Dove gira: XDP	4
1.3	Le tre pipeline	4
2	La matematica che sta dentro il kernel	5
2.1	Perché i numeri sono interi	5
2.2	Il problema dei bias, e l'errore che ci stava dentro	5
2.3	Il TTL: dividere il prodotto, mai il valore	6
3	Le classi non sono porte	7
3.1	Il problema	7
3.2	La soluzione: una catena esplicita	7
3.3	Un errore che questa separazione ha fatto emergere	8
4	Il motore non è lo scenario	9
4.1	Com'era	9
4.2	Com'è adesso: tre strati	9
4.3	La prova	10
5	Il modello non è quello depositato	11
5.1	La domanda	11
5.2	I modelli sintetici	11
5.3	Che cosa era già provato, e che cosa no	11
5.4	Il confronto a quattro vie	12
5.5	Il risultato	12
5.6	Ma ci è arrivato trovando due difetti	13
5.7	La correzione	13
5.8	Che cosa è costata, e la prova che regge	14
5.9	Tre difetti, e chi li ha trovati	14
5.10	Che cosa questo capitolo dimostra	15
6	Dall'emulatore al kernel vero	16
6.1	Che cosa provavano i test di prima	16
6.2	Il banco nuovo	16
6.3	Tre difetti del test, trovati misurando	16
6.4	Il risultato	17
7	La feature che non c'era	18
7.1	Il sintomo	18
7.2	Perché nessun test lo vedeva	18
7.3	La correzione	18
7.4	La prova che adesso è viva	19
7.5	Il prezzo, misurato	19

8	Il nodo non sapeva di essere un nodo	20
8.1	L'ingresso più grande di tutti	20
8.2	Il sintomo	20
8.3	La correzione, e perché una tabella e non un array	20
8.4	Una pipeline ha resistito un giorno	20
8.5	La prova che il nodo è vivo	21
9	Il verificatore non conta le istruzioni	22
9.1	Due monete diverse	22
9.2	Il ciclo era girato dalla parte sbagliata	22
9.3	Perché il risultato non cambia di un bit	22
9.4	Il risultato, che sembra un errore	22
9.5	Un effetto collaterale che vale una riga di tabella	23
10	I numeri, e che cosa dicono davvero	24
10.1	La tabella	24
10.2	Primo risultato: la dimensione non predice la velocità	24
10.3	Secondo risultato: quanto valgono le latenze assolute	25
10.4	Terzo risultato: la genericità ha un costo di verifica	26
10.5	Quarto risultato: togliere il compilatore dal nodo	26
10.6	Che cosa questi numeri non dicono	26
11	Come cresce il costo	28
11.1	La domanda	28
11.2	Come è impostata la misura	28
11.3	Profondità: chi paga in spazio e chi in tempo	29
11.4	Aggiornare il modello: due ordini di grandezza	30
11.5	Larga o profonda? Dipende da chi la esegue	30
11.6	Congelare il nodo: conviene?	32
11.7	Congelare quali porte esistono	36
11.8	Due larghezze, due risposte opposte	38
11.9	I pesi contano, ma solo per una delle tre	40
11.10	Il risultato che non cercavamo	40
11.11	Che cosa questi grafici non dicono	41
12	Il tempo fra arrivo e partenza	42
12.1	La domanda	42
12.2	Due banchi che misurano due cose diverse	42
12.3	L'architettura del banco, e che cosa si misura dove	43
12.4	Il banco	44
12.5	Tre cose che rendevano il confronto ingiusto	44
12.6	Misurare arrivo → partenza	45
12.7	Il risultato: la latenza	45
12.8	Il risultato che non regge: la portata	46
12.9	Un tentativo che non ha funzionato	46
12.10	Le stesse pipeline, quaranta volte più lente	47
12.11	Perché il confronto regge lo stesso	48
12.12	Che cosa questi numeri non dicono	49

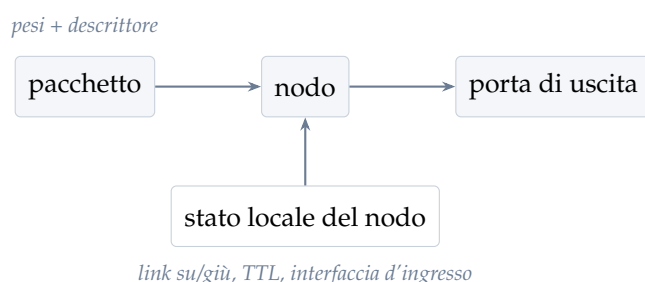
13	Quel che resta	50
13.1	Fatto	50
13.2	Aperto	51
13.3	Il filo conduttore	51
14	I comandi	53
14.1	Correttezza: il modello fa quello che deve	53
14.2	Kernel: si carica, e quanto costa	53
14.3	Datapath reale: il pacchetto attraversa davvero	54
14.4	Analisi parametrica: come cresce il costo	54
14.5	Throughput e latenza end-to-end	55
14.6	Costi specifici	55
14.7	Oggetto precompilato (AOT)	55
14.8	Diagnostica, quando qualcosa non torna	56
14.9	L'ordine consigliato	56

Di che cosa stiamo parlando

1.1 L'idea in una pagina

Un router normale decide dove mandare un pacchetto consultando una tabella riempita da un protocollo di routing. L'idea di **IPA** (*Intelligent PAckets*) è diversa: il pacchetto **porta con sé** una piccola rete neurale, e ogni nodo che attraversa la esegue per decidere l'uscita.

Niente segnalazione, niente tabella condivisa: la decisione nasce dai dati che il nodo ha sotto mano in quel momento.



Il modello è una rete piccola: quattro tipi di ingresso, due strati nascosti da quattro neuroni, sette uscite. In tutto **319 pesi**. Deve stare in un header ed essere eseguibile in decine di nanosecondi.

1.2 Dove gira: XDP

Il codice gira in **eBPF** agganciato a **XDP**, il punto più precoce in cui il kernel Linux permette di toccare un pacchetto: prima che venga costruita la struttura `sk_buff`, quindi prima di quasi tutto il resto dello stack di rete.

Questo impone un vincolo forte, che spiega metà delle scelte di progetto: un programma eBPF deve essere **dimostrato sicuro** da un componente del kernel chiamato *verificatore* prima di poter girare. Niente cicli di lunghezza ignota, niente accessi non controllati alla memoria, niente ricorsione. Tutto ciò che nel documento sembra contorto — cicli srotolati, tetti dichiarati a priori, tabelle al posto di calcoli — nasce da lì.

1.3 Le tre pipeline

Lo stesso modello è implementato in tre modi, per misurare un compromesso.

	i pesi stanno	cambiare modello
P1 hardcoded	scritti nel codice C	ricompilare
P2 template	in una mappa	scrivere la mappa
P3 modular	in una mappa, uno strato per programma	scrivere la mappa

P1 è veloce perché il compilatore vede i pesi e li semplifica; è rigido perché ogni modello nuovo richiede una ricompilazione. P2 e P3 sono l'opposto. Il lavoro di settembre serve a rendere questo confronto *misurabile davvero*.

La matematica che sta dentro il kernel

2.1 Perché i numeri sono interi

Nel kernel non ci sono numeri in virgola mobile: eBPF lavora con interi. Il modello però è addestrato in virgola mobile. Serve una conversione, e si chiama **quantizzazione**.

Ogni peso reale w diventa un intero a 8 bit:

$$w_{\text{int}} = \text{round}(w \cdot s)$$

dove s è il **fattore di scala**, uno solo per tutta la rete.

Perché il risultato stia in un byte con segno serve

$$s \cdot \max_i |w_i| \leq 127.$$

Sul modello depositato $\max |w| = 5,098$, quindi $s \leq 24,9$. Due scelte convivono nel progetto: $s = 24$ (il massimo intero ammesso) e $s = 16$ (la potenza di due più vicina per difetto, che rende le divisioni degli shift).

Superare quel limite non dà un errore: dà un *overflow silenzioso*. I pesi grandi si troncano e il modello sbaglia senza che nulla lo segnali. Per questo i test distinguono le scale *rappresentabili* da quelle che saturano, e misurano solo sulle prime.

2.2 Il problema dei bias, e l'errore che ci stava dentro

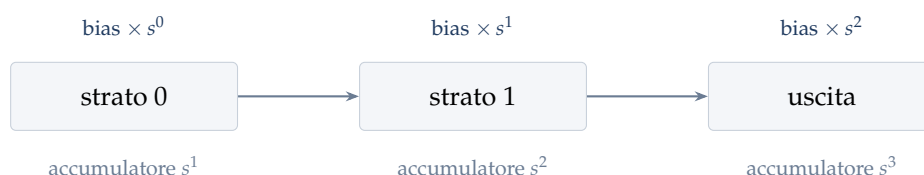
Qui sta l'errore più sottile trovato in settembre, e vale la pena seguirlo passo passo perché è istruttivo.

Quando moltiplichiamo un ingresso per un peso quantizzato, il risultato porta un fattore s in più rispetto al valore reale:

$$x \cdot w_{\text{int}} = x \cdot w \cdot s.$$

Se impiliamo gli strati, quel fattore si accumula. Dopo $\ell + 1$ moltiplicazioni l'accumulatore dello strato ℓ porta $s^{\ell+1}$.

Ma il **bias** è memorizzato come un peso qualsiasi, cioè con un solo fattore s . Sommarlo così com'è significa sommare una quantità sistematicamente troppo piccola. Va riportato nelle unità dell'accumulatore:



La regola è semplice: allo strato ℓ (contando da zero) il bias va moltiplicato per s^ℓ .

L'errore

Il modello di riferimento in Python — quello con cui si controlla che il datapath calcoli la cosa giusta — applicava s^0 e s^1 correttamente ai primi due strati, e lasciava l'**ultimo strato** a s^0 invece di s^2 .

Il bias dell'uscita era sottopesato di $s^2 = 24^2 = 576$.

Perché nessuno se n'era accorto per mesi

Tutti i test eseguivano il modello con **tutti i link attivi**. In quella condizione gli accumulatori sono grandi e un errore di 576 non sposta mai il massimo. Basta spegnere qualche link perché gli accumulatori si riducano e l'errore decida l'esito.

Nello stesso file c'era una seconda funzione di riferimento, scritta per forme arbitrarie, che usava la formula generale s^ℓ ed era **già corretta**. Due riferimenti, uno giusto e uno sbagliato, e quello sbagliato era l'unico mai esercitato con link spenti.

Corretto l'errore, il riferimento e le tre pipeline concordano su tutti i casi provati.

La lezione non è "c'era un bug". È che **un test che varia un solo ingresso non è un test**: verifica una condizione e la ripete. Ci torneremo nel capitolo 5, dove lo stesso schema si ripresenta in una forma ancora più netta.

2.3 Il TTL: dividere il prodotto, mai il valore

Il modello è addestrato con il TTL *normalizzato*, cioè ttl/T con $T = 30$ il valore iniziale. Con i numeri interi

$$\frac{\text{ttl}}{30} = 0 \quad \text{per ogni } \text{ttl} < 30,$$

e la feature sparirebbe. La divisione va quindi applicata al *prodotto*:

$$\left\lfloor \frac{\text{ttl} \cdot w_{\text{int}}}{T} \right\rfloor \quad \text{e non} \quad \left\lfloor \frac{\text{ttl}}{T} \right\rfloor \cdot w_{\text{int}}.$$

Nel C generato si legge esattamente così:

```
(((__s64)_ttl * 11LL) / 30LL)
```

Le classi non sono porte

3.1 Il problema

La rete produce sette numeri; il più grande vince. Ma *che cosa significa* la classe vincente?

All'inizio di settembre la risposta era sparsa e incoerente. In punti diversi del progetto convivevano quattro convenzioni su quale classe significasse "scarta il pacchetto":

Convenzione	Dove
la classe 6 è DROP	in un punto
la classe 5 è DROP	in un altro
$\text{DROP} = n_{\text{out}} - 1$	in un terzo
$\text{DROP} = \max(\text{etichette}) + 1$	dedotto dai dati

E sotto ce n'era una quinta, implicita e più insidiosa: che la classe k volesse dire "manda fuori dall'interfaccia k ". Funzionava per caso, perché nel modello depositato le porte sono numerate come le classi.

3.2 La soluzione: una catena esplicita

Adesso il significato è **dichiarato**, mai dedotto, e passa per una catena di quattro passi, ognuno con un responsabile diverso:



- **classe**: quello che il modello ha calcolato;
- **azione**: FORWARD, DROP o UNUSED — una classe mai addestrata esiste ed è dichiarata tale;
- **porta logica**: un numero che il *modello* conosce;
- **ifindex**: l'interfaccia vera, che solo il *nodo* conosce e che è diversa su ogni macchina.

Nel modello depositato la dichiarazione è:

classe	azione	porta logica
0-4	FORWARD	0-4
5	DROP	—
6	UNUSED	—

Il punto non è che adesso funziona: funzionava anche prima, per coincidenza numerica. Il punto è che adesso **smetterebbe di funzionare rumorosamente** se il modello cambiasse, invece di continuare a girare dando risposte plausibili e sbagliate.

3.3 Un errore che questa separazione ha fatto emergere

Il datapath teneva un contatore per classe, ma scriveva nella casella della *porta* invece che in quella della *classe*. Sembrava giusto perché le due numerazioni coincidono nel modello depositato.

Con un modello in cui non coincidono, il contatore avrebbe misurato la cosa sbagliata in silenzio. Separare i due spazi di indici ha reso l'errore visibile e correggibile.

Il motore non è lo scenario

4.1 Com'era

Il progetto era nato attorno a una topologia precisa: **Germany50**, una rete di 50 router usata come riferimento in letteratura, con due host attaccati, emulata con **Kathara** (container Docker connessi a formare la rete).

Due numeri erano ovunque nel codice:

$$n_{\text{nodi}} = 52, \quad n_{\text{interfacce}} = 6.$$

Il primo è $50 + 2$. Il secondo è il **grado massimo** della rete: il nodo con più vicini. Non è il grado di un nodo qualsiasi — tutti gli altri riempiono di zeri gli slot che non usano.

Questi numeri determinano la larghezza dell'ingresso:

$$n_{\text{in}} = \underbrace{6}_{\text{link}} + \underbrace{6}_{\text{iface}} + \underbrace{1}_{\text{ttl}} + \underbrace{52}_{\text{nodo}} = 65.$$

Il problema non era che esistessero, ma che il motore li conoscesse. C'era un *default* interno: se nessuno diceva niente, il motore assumeva 6 e 52. Ogni chiamante che dimenticava di specificare lo scenario risultava silenziosamente corretto per Germany50 e silenziosamente sbagliato altrove.

4.2 Com'è adesso: tre strati



La dipendenza punta **solo verso l'alto**. Il motore non sa che cosa sia Germany50.

Le dimensioni si risolvono in ordine: una variabile d'ambiente, poi un file di sistema, poi il blocco `trained_on` del descrittore del modello. Se nessuna delle tre risponde, il motore **si ferma** e dice dove ha cercato.

Un default comodo è il modo più efficace di nascondere un'assunzione. Preferire un errore esplicito a un valore plausibile è una scelta di progetto, non pignoleria: il valore plausibile non si manifesta mai come errore, solo come risultato sbagliato mesi dopo.

4.3 La prova

Non basta dirlo. Il motore genera oggi un datapath corretto per cinque topologie che non hanno niente a che vedere con quella d'origine:

iface	nodì	strati	n_{in}	n_{out}	vettore link
3	8	4-4	15	4	v[3]
4	16	6	25	5	v[4]
5	24	4-4-4	35	6	v[5]
2	6	3-3	11	3	v[2]
6	52	4-4	65	7	v[6]

Nessuna riga di C scritta a mano cambia fra una riga e l'altra.

Il modello non è quello depositato

5.1 La domanda

Il capitolo precedente ha tolto la topologia dalle assunzioni del codice. Restava dentro un'assunzione più silenziosa: **un solo modello**. Ogni misura, ogni confronto fra pipeline e ogni prova di correttezza girava sullo stesso checkpoint fornito dal relatore — 65-4-4-7, scala 25.

Il rischio non è teorico. Un datapath che calcola la cosa giusta su un modello può calcolarla per caso: perché quel modello ha una forma particolare, perché i suoi pesi cadono in un intervallo comodo, o perché una costante scritta nel codice coincide con un suo parametro. Finché il modello è uno solo, la differenza fra “corretto” e “corretto qui” non si vede.

5.2 I modelli sintetici

La risposta sono **sette scenari generati**, con pesi casuali e nessun rapporto con il checkpoint depositato. Coprono un asse ciascuno:

Scenario	Forma	Pesi
ones	11-2-3	33
small	15-4-4	84
mixed	18-6-5	149
ipa_like	65-4-4-7	319
sparse	65-4-4-7	319
deep	65-4-4-4-7	339
large	117-8-8-9	1 097

Il generatore non produce solo numeri a caso: rispetta la **struttura** delle feature, che è la parte che un datapath può sbagliare. Ogni feature dichiara il suo tipo — *real* (un valore in un intervallo, che il datapath porta come intero diviso per una scala), *integer*, *binary* (un vettore di 0/1), *onehot* (esattamente un 1) — con i suoi estremi, la sua scala e i suoi vincoli. Ogni scenario porta con sé **1 000 vettori d'ingresso**, nella versione intera e in quella a virgola mobile, e le predizioni attese per entrambe.

Gli artefatti non stanno in git: sono rigenerabili byte per byte a parità di seme, e il test li rigenera da solo se non li trova.

5.3 Che cosa era già provato, e che cosa no

Su quei sette scenari il progetto verificava che i pesi **compilassero** e passassero il verificatore del kernel, e che i control plane accettassero i blocchi. È una prova reale ma di una cosa sola: che il programma esiste. Non dice *quale classe sceglie*.

La distinzione che mancava

Un programma che compila e gira può comunque decidere diversamente dal modello che dovrebbe eseguire. La domanda scoperta era: *a parità di ingresso, il datapath sceglie la stessa classe del riferimento?*

5.4 Il confronto a quattro vie

Due confronti non bastano, perché un disaccordo ha più di una causa possibile e le cause chiedono rimedi diversi. Le vie sono quattro:

Confronto	Che cosa isola
float vs int8 (Python)	l'errore di quantizzazione : il modello a interi decide diversamente da quello a virgola mobile. Può legittimamente non essere 100%.
int8 (Python) vs int8 (riferimento)	controllo : due implementazioni intere indipendenti del progetto devono concordare fra loro, o non è chiaro quale sia il riferimento.
int8 (riferimento) vs int8 (eBPF)	se il datapath è esatto rispetto alla configurazione che gli è stata data .
int8 (Python) vs int8 (eBPF)	l'errore di implementazione . Deve valere 100%: un solo disaccordo qui è un difetto del datapath.

Perché non i 1 000 vettori depositati

Il datapath non accetta un vettore d'ingresso arbitrario: lo *costruisce* da ciò che ha sotto mano — le mappe dense, il TTL del pacchetto, la porta d'ingresso risolta, l'indice del nodo. Un one-hot node diverso a ogni campione vorrebbe dire ricompilare il programma a ogni campione.

Si campiona quindi nello spazio che il datapath sa davvero esprimere, e il vettore Python si costruisce allo stesso modo. Confrontare due implementazioni su ingressi che una delle due non potrebbe mai ricevere non direbbe niente su nessuna delle due.

5.5 Il risultato

Trecento ingressi per scenario, seme fisso, pipeline P1.

Scenario	float vs int8	int8 Python vs eBPF	decisioni
ones	100,00 %	100,00 %	300/300
small	98,00 %	100,00 %	300/300
mixed	100,00 %	100,00 %	300/300
ipa_like	97,67 %	100,00 %	300/300
sparse	99,33 %	100,00 %	300/300
deep	98,67 %	100,00 %	300/300
large	99,67 %	100,00 %	300/300

Sette scenari su sette, equivalenza numerica esatta. La colonna di sinistra — fra 97,67 % e 100 % — è la quantizzazione, cioè una proprietà del modello e della scala scelta, non un difetto.

5.6 Ma ci è arrivato trovando due difetti

Il valore del test non sta nella tabella: sta nel fatto che la prima esecuzione **non** dava 7/7. Dava 1/7, e le due cause erano di natura diversa.

Il primo: il banco

Cinque scenari su sette stavano fra il 71 % e l'85 %. Il banco non comunicava al kernel la porta d'ingresso: `prog_test_run` accetta un parametro `ingress_ifindex` ma lo **ignora** — gira senza `ctx_in`, e il programma vede sempre l'`ifindex` del dispositivo di prova del kernel. Il riferimento Python usava una porta variabile, il datapath ne vedeva sempre nessuna, e le due metà costruivano due ingressi diversi.

Il segnale diagnostico è stato più utile del sintomo: dopo la prima correzione le percentuali non si erano mosse **di un centesimo**. Codice che non cambia niente non è codice sbagliato: è codice mai eseguito. Confermato senza kernel, simulando “il datapath vede porta 0”: le percentuali osservate si riproducono con scarto **0,00** su sei scenari.

Il secondo: la scala compilata

Resisteva `small`, al 70,33 %. Con una coincidenza che era la risposta: le righe *int8 Python vs riferimento* e *int8 Python vs eBPF* davano lo stesso identico numero, e *riferimento vs eBPF* dava 100 %. Il datapath non sbagliava un conto: eseguiva **esattamente** una configurazione sbagliata.

Tutte e tre le pipeline fissavano la scala della feature `ttl` a **30** in fase di compilazione — `feature_scale()` in P1, `T2_TTL_SCALE` in P2, `ML_TTL_SCALE` in P3 — e la struttura del descrittore letto a runtime, `struct feat_ent`, non aveva un campo dove metterla. Un modello addestrato su `ttl/16` veniva eseguito con `ttl/30`. Gli scenari `small` (16), `large` (64) e `ones` (8) sono esattamente quei modelli.

Sul checkpoint depositato, che usa 30, il difetto è invisibile.

La genericità di P2 e P3 — “il descrittore si legge a runtime, quindi qualunque modello va bene” — era vera per la forma, il numero di feature e gli offset, e **falsa per la scala**: l'unico numero del descrittore che deve seguire il modello era l'unico compilato dentro. Un'affermazione di genericità va verificata voce per voce, non in blocco.

5.7 La correzione

La scala ora viaggia nel descrittore. Il campo c'era già: `struct feat_ent` aveva un byte di riempimento inutilizzato, e le scale in gioco (8, 16, 30, 64) ci stanno.

Dove	Che cosa cambia
<code>model_meta</code>	<code>feature_scale_of(feat)</code> risponde per <i>feature</i> e non per <i>tipo</i> ; la scala entra nel descrittore risolto.
P1	compila la scala del descrittore invece di quella del catalogo.
P2 / P3	<code>_pad</code> diventa <code>scale</code> ; la divisione la legge dalla mappa. Con 0 si ricade sul default compilato, quindi i descrittori scritti prima restano validi.

Il modello depositato risolve a 30 come prima: nessuna regressione.

Un effetto collaterale che il compilatore ha fermato

Passare da una costante a una variabile ha rotto la compilazione di P2: *“unsupported signed division”*. Finché il divisore era un `#define` clang lo trasformava in uno spostamento di bit; da variabile diventa una divisione vera, e **BPF non ha la divisione con segno**. Il numeratore qui ce l’ha, perché i pesi possono essere negativi.

La correzione divide il valore assoluto senza segno e rimette il segno, con troncamento verso lo zero — lo stesso arrotondamento del riferimento Python, perché sui negativi le due implementazioni coincidono solo se arrotondano allo stesso modo.

5.8 Che cosa è costata, e la prova che regge

Una correzione che tocca il codice che gira per ogni pacchetto va pagata e misurata, non solo dichiarata. La suite kernel, rieseguita sul modello depositato:

Pipeline	istruzioni prima	dopo	differenza	
baseline	155	155	invariata	
P1 hardcoded	1 026	1 026	invariata	
P2 template	14 628	14 985	+357	(+2,4 %)
P3 modular	12 031	12 349	+318	(+2,6 %)

Il costo è **due virgola cinque per cento** sulle due pipeline generiche: il codice della divisione a divisore variabile più la lettura della scala dalla mappa. Che baseline e P1 restino *identiche al conteggio* è la verifica che il cambiamento non ha toccato percorsi che non doveva toccare — P1 compila la scala come prima, solo presa da un posto diverso.

Perché la latenza non si può usare per giudicare

Nello stesso run la latenza si è spostata su tutte e quattro le colonne, **compresa la baseline**: da 29 a 35 ns, cioè +21 %, su un programma che non è stato modificato di una riga.

Quel +21 % è il metro del rumore della macchina in quell’esecuzione, e sta *sopra* lo spostamento di P2 (+9 %) e P3 (+5 %). Le differenze di latenza qui non sono attribuibili alla modifica: l’unica colonna che dice qualcosa è quella delle istruzioni, che è deterministica.

L’esito funzionale è kernel suite: PASS — le quattro pipeline, la gestione del TTL, il reroute su link caduto, le architetture alternative e i due modelli concorrenti registrati insieme su P2 e su P3. E nel registro del descrittore si legge la scala arrivare a destinazione:

```
model_desc[0] = feats=[
(1,6,0,1) (2,6,6,1) (3,1,12,30) (4,52,13,1)
```

Il quarto campo è la scala: `ttl` porta 30 fin dentro la mappa, e non è più un numero scritto nel sorgente.

5.9 Tre difetti, e chi li ha trovati

Vale la pena tenere il conto, perché dice qualcosa sul metodo più che sul codice.

Difetto	Di chi	Trovato da
Porta d'ingresso non detta al kernel	banco	il confronto, al primo run
Scala del TTL compilata a 30	datapath	il confronto, tolto il primo
Divisione con segno, non ammessa da BPF	correzione	il compilatore
Funzione nel ramo sbagliato di <code>#ifndef</code>	correzione	il compilatore, al secondo giro

Gli ultimi due sono stati introdotti *riparando* il secondo, e nessuno dei due sarebbe stato visibile senza compilare davvero. Da lì è nato `check_source_visibility.py`: i sorgenti eBPF di questo progetto non sono file ma stringhe, concatenate in combinazioni diverse dai chiamanti e ritagliate dal preprocessore con macro diverse. Una funzione può stare prima del suo uso *nel file* e sparire lo stesso nella combinazione che conta. Il controllo valuta le guardie e verifica la visibilità in tutte e cinque le combinazioni realmente compilate — senza root e senza BCC, quindi prima di spedire e non dopo.

5.10 Che cosa questo capitolo dimostra

Che le tre pipeline calcolano ciò che il modello dice, su sette modelli che nessuno ha scelto per farle funzionare. E, di passaggio, che l'affermazione era falsa prima di essere verificata — il che è il motivo per cui la verifica serviva.

Dall'emulatore al kernel vero

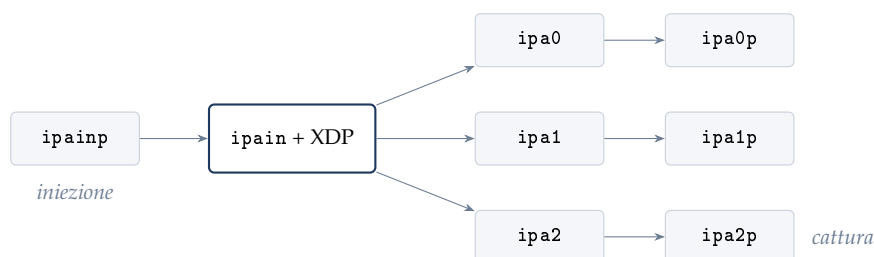
6.1 Che cosa provavano i test di prima

Fino a settembre ogni misura veniva da una chiamata di sistema, `BPF_PROG_TEST_RUN`: si dà al programma un buffer, lo si esegue N volte, e il kernel riporta quanti nanosecondi in media.

È utile e onesto per confrontare le tre pipeline fra loro. Ma non tocca un'interfaccia, non ha un driver, non consegna niente. Il programma poteva dire "redirigerei sulla porta 3" e nessuno verificava che un pacchetto ne uscisse davvero.

6.2 Il banco nuovo

Kathara è stato sostituito da qualcosa di più semplice e più vicino al vero: coppie di interfacce virtuali (veth) create sull'host.



Si inietta un frame da un capo, il programma XDP gira davvero, e si legge da quale interfaccia è uscito. Questo risponde alla domanda che `BPF_PROG_TEST_RUN` non può porre.

Due dettagli che sembrano minori e non lo sono

Modalità di attacco. XDP si può agganciare in due modi: *native*, dentro il driver, prima che esista la `sk_buff`; oppure *generic*, dopo. Il progetto usava sempre *generic*, e il motivo era scritto nel codice: serviva per le interfacce dentro i container dell'emulatore. Tolto l'emulatore, quel motivo cade. Adesso il default è *native*, la modalità *davvero in effetto* viene riletta dal kernel dopo l'aggancio, e un fallimento è un errore, non una retrocessione silenziosa.

Il vincolo delle veth. Redirigere verso una veth funziona solo se il lato ricevente ha a sua volta un programma XDP agganciato: il kernel alloca le code necessarie solo in quel caso. Senza, il redirect fallisce in silenzio e il pacchetto sparisce — indistinguibile da un modello che decide di scartare tutto. Il banco aggancia quindi un programma minimo a ogni capo.

6.3 Tre difetti del test, trovati misurando

Il banco nuovo ha trovato più difetti nei *test* che nel prodotto. Vale la pena elencarli, perché il motivo per cui erano invisibili è sempre lo stesso.

1. Il TTL non muove questo modello

La prima versione del test variava il TTL da 2 a 8 e riportava *"7 verifiche su 7 superate"*. Misurando lo spazio raggiungibile si scopre che con tutti i link attivi il modello risponde **sempre classe 2**, per qualsiasi TTL. Quel 7 su 7 era **un caso solo, ripetuto sette volte**.

È `link_state` la feature che sposta la decisione. Il test adesso *cerca* un ingresso per ogni classe, invece di spazzare alla cieca.

2. Un pacchetto del kernel contato come risultato

Su una veth appena creata il kernel manda del traffico suo (IPv6 multicast). Una volta è stato catturato e contato come la risposta della pipeline. È passato per caso, perché era sulla porta giusta.

Adesso la cattura filtra sui campi che il datapath non riscrive, e IPv6 viene disabilitato sulle interfacce del banco.

3. La porta 0 era invisibile per costruzione

L'ingresso era anche la porta logica 0. Una classe che inoltrava su quella porta rimandava il frame fuori dall'interfaccia da cui era entrato — dove la cattura non lo distingue da quello appena iniettato. Una decisione *corretta* sulla porta 0 non era osservabile.

Il banco ha adesso un ingresso dedicato, separato da tutte le porte.

Un test che non può fallire non è un test. In tutti e tre i casi il risultato era verde e privo di contenuto, e in tutti e tre il difetto è emerso solo mettendo in mezzo un kernel vero.

6.4 Il risultato

Tutte e tre le pipeline superano oggi la verifica end-to-end su un datapath reale: XDP native, redirect vero, pacchetto ripreso sulla porta d'uscita, e scarto verificato dal *contatore* e non dal silenzio — perché "non è arrivato niente" è anche ciò che si osserva quando il redirect si perde.

La feature che non c'era

7.1 Il sintomo

Il modello ha quattro ingressi. Uno di questi è *da quale interfaccia è arrivato il pacchetto*, codificato *one-hot*: un vettore di sei posizioni con un solo uno.

Il datapath leggeva l'indice dell'interfaccia dato dal kernel — l' *ifindex* — e lo usava direttamente come posizione nel vettore, con un controllo di validità:

$$1 \leq \text{ifindex} \leq n_{\text{interfacce}}.$$

Gli *ifindex* però li assegna il kernel, in ordine di creazione. Su un container appena avviato valgono 2, 3, 4. Su una macchina che ha creato qualche interfaccia virtuale valgono 205, 217, 229.

Il controllo falliva sempre. **Nessun bit veniva acceso.** Il modello girava di fatto su tre ingressi invece di quattro.



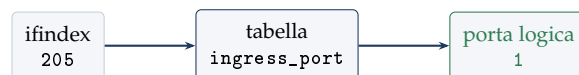
7.2 Perché nessun test lo vedeva

Perché nessun test controllava che quella feature *contribuisse* qualcosa. Riferimento e datapath la ignoravano entrambi, quindi **concordavano** — e l'accordo veniva registrato come successo.

Una pipeline aveva in effetti una tabella di traduzione, ma compilata dentro il programma come [2, 3, 4, 5, 6, 7]: la stessa assunzione, scritta in un altro modo. Le altre due usavano l'indice grezzo. Le tre pipeline sceglievano quindi *colonne diverse* per lo stesso pacchetto — una discrepanza documentata in un commento e mai risolta.

7.3 La correzione

La traduzione è diventata una **tabella a runtime**, riempita dal piano di controllo leggendo le interfacce del nodo:



È il **simmetrico** della tabella già usata in uscita: là "quale interfaccia realizza la porta che il modello ha scelto", qui "quale porta è l'interfaccia da cui il pacchetto è arrivato". Entrambe sono fatti del nodo, entrambe si risolvono quando il nodo esiste.

7.4 La prova che adesso è viva

Prima e dopo la correzione, gli ingressi che raggiungono ciascuna classe **cambiano**:

classe	prima (feature morta)	dopo (feature viva)
0	100110, ttl 27	100111, ttl 20
1	010000, ttl 2	010000, ttl 14
5	000000, ttl 2	000000, ttl 12

Stesso stato dei link, TTL diverso per ottenere la stessa classe: l'ingresso in più contribuisce. E il TTL ora sposta la decisione, cosa che prima non faceva.

7.5 Il prezzo, misurato

Una lettura di tabella in più per pacchetto:

	prima	dopo
letture per pacchetto (P1)	4,0	5,0
letture per pacchetto (P3)	26,0	28,0
istruzioni (P1)	997	971

Le istruzioni di P1 *calano*: la tabella ha sostituito un blocco switch a sei casi, e una lettura costa meno di un salto.

Una lettura in più arriverà poco dopo, per la stessa ragione applicata a un altro ingresso: è il capitolo che segue. Le cifre finali sono 6,0 letture per P1 e 29,0 per P3.

Il nodo non sapeva di essere un nodo

8.1 L'ingresso più grande di tutti

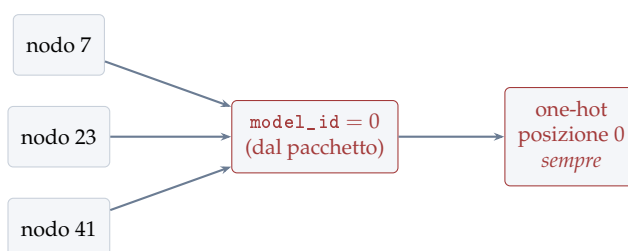
Dei quattro ingressi del modello, quello che pesa di più è *quale nodo sta elaborando il pacchetto*. Su Germany50 sono 52 nodi, quindi un vettore one-hot di 52 posizioni: **208 dei 319 pesi** della rete, quasi due terzi.

8.2 Il sintomo

Il datapath riempiva quel vettore con il campo `model_id` preso dall'intestazione IPA del pacchetto.

Sono due cose diverse. `model_id` dice *quale modello* il pacchetto chiede di usare; il vettore vuole sapere *quale nodo* lo sta elaborando. E il `model_id` non cambia lungo il percorso: è scritto dal mittente e viaggia identico fino alla destinazione.

Con un solo modello registrato — il caso normale — **ogni nodo della rete accendeva la stessa identica posizione**. Due terzi dei pesi della rete non distinguevano nulla.



8.3 La correzione, e perché una tabella e non un array

Stessa forma della correzione precedente: una **tabella letta a runtime**, riempita quando il nodo viene messo in servizio, dalla variabile d'ambiente `IPA_NODE_ID` o dall'indice del nodo nella topologia.

Un dettaglio che sembra tecnico e non lo è: la tabella è di tipo *hash* e non *array*. Un array in questo ambiente è preallocato e riempito di zeri, quindi una lettura riesce **sempre**: "non configurato" e "sono il nodo 0" tornerebbero lo stesso valore, indistinguibili. Sarebbe esattamente il difetto che la tabella serve a togliere. Con un hash, finché nessuno scrive non c'è voce: il vettore resta spento, ed è la risposta onesta.

Questo errore è stato commesso e corretto: la prima versione usava un array, e un test su un'architettura alternativa è fallito 5 volte su 5 prima che se ne capisse il motivo.

8.4 Una pipeline ha resistito un giorno

P1 e P2 hanno accettato la modifica subito. P3 no: il verificatore del kernel ha **rifiutato il programma**, dodici configurazioni di fila.

Il motivo è controintuitivo e vale la pena averlo scritto. Il verificatore non misura quanto è lungo il programma: esplora i *cammini* possibili, e il numero di stati che deve tenere è il **prodotto** dei valori indipendenti che segue dentro un ciclo srotolato, non la loro somma.

$$\text{stati} \sim v_1 \times v_2 \times \cdots \times v_k$$

L'unica versione che caricava era quella in cui l'indice del nodo *non era un valore indipendente*: `nodo = model_id`, cioè lo stesso registro che il verificatore stava già seguendo. Un valore, non due.

Escluse una a una, ognuna con la sua prova: il tipo di tabella, la sua dichiarazione, il valore, il tipo del dato, il numero di letture, la larghezza in bit. Nessuna era la causa.

Quello che ha liberato spazio non toccava il nodo: un **tetto** di tutt'altra feature. Il programma teneva in vita otto posizioni per un vettore di occupazione delle code che il modello depositato *non dichiara nemmeno*. Portato quel tetto da 8 a 1, il programma ha caricato — e più piccolo di prima.

Questi limiti sono **scogliere, non pendenze**. Nello stesso giro, ridurre *anche* un secondo tetto — cioè chiedere *meno* — ha fatto fallire di nuovo il caricamento. Un tetto qui si cambia e **si rimisura**; non si ragiona.

8.5 La prova che il nodo è vivo

Stesso programma compilato, due esecuzioni che differiscono solo per la tabella del nodo:

esecuzione	esito sul pacchetto
senza indice di nodo	scartato
indice di nodo = 7	non scartato

Nient'altro è cambiato fra le due. L'ingresso sposta la decisione.

Il verificatore non conta le istruzioni

9.1 Due monete diverse

Il tetto sulle code portato a 1 era una perdita vera: a 1, un modello che dichiara davvero quella feature viene *rifiutato*. Per recuperarlo serviva margine di verifica, e il margine è arrivato da una riscrittura che non toglie niente.

9.2 Il ciclo era girato dalla parte sbagliata

Il primo strato della rete si calcola così: per ogni neurone, si sommano i contributi di ogni feature dichiarata nel descrittore.

Il codice lo faceva nell'ordine ovvio — neuroni fuori, feature dentro:

```
per ogni neurone j (8 volte)
  per ogni feature f (4 volte)
    di che tipo e' f?      <- catena a 5 rami
    di che larghezza e' f? <- controllo per ogni posizione
  accumula
```

Ma *di che tipo* e *di che larghezza* è una feature non dipende dal neurone. Quella catena a cinque rami veniva quindi rivalutata $8 \times 4 = 32$ volte per pacchetto, e il controllo sulla larghezza di un vettore denso $8 \times 8 = 64$ volte, per ottenere ogni volta la stessa risposta.

Girato al contrario — feature fuori, neuroni dentro — diventa:

```
per ogni feature f (4 volte)
  di che tipo e' f?      <- 4 volte, non 32
  di che larghezza e' f? <- 8 volte, non 64
  per ogni neurone j (8 volte)
    accumula              <- nessun ramo qui dentro
```

9.3 Perché il risultato non cambia di un bit

Cambia solo l'ordine in cui i termini vengono sommati. I termini sono gli stessi, calcolati allo stesso modo. E la somma fra interi a 64 bit in complemento a due è associativa e commutativa — anche in caso di traboccamento, che si comporta identicamente nei due ordini:

$$(a + b) + c = a + (b + c).$$

Ogni accumulatore finisce quindi **bit per bit identico**. Non "circa uguale": identico.

9.4 Il risultato, che sembra un errore

	prima	dopo
P2, programma principale	16 144	14 587
P3, primo strato	6 417	6 850
P3, tetto sulle code caricabile	1	8

P3 è **cresciuto** di 433 istruzioni ed è diventato molto più economico da verificare. Prima della riscrittura, col tetto delle code a 8 non caricava niente; dopo, carica a 10 207 istruzioni.

Dimensione del programma e complessità di verifica sono **monete diverse**. Il verificatore paga i cammini.

9.5 Un effetto collaterale che vale una riga di tabella

Contare quante letture di tabella fa un programma per pacchetto richiede di ricompilarlo con un contatore su ogni punto di lettura. Le versioni strumentate di P2 e P3 **sforavano il verificatore**: per questo la riga "letture di tabella" riportava *n.d.* per entrambe.

Con il margine liberato dalla riscrittura, tutte e due caricano. La misura che era impossibile è diventata una riga di tabella — 11 letture per P2, 29 per P3 — senza che nessuno abbia riscritto il contatore.

I numeri, e che cosa dicono davvero

10.1 La tabella

Misure su una macchina virtuale a 4 processori, modello 65-4-4-7, scala 24.

	baseline	P1	P2	P3
istruzioni caricate	155	1 026	14 628	12 031
salto fra programmi	0	1	1	3
letture di tabella	3,0	6,0	11,0	29,0
memoria delle tabelle	280 B	2 356 B	10 416 B	19 508 B
latenza (minimo di 7)	29 ns	73 ns	262 ns	421 ns

Il *baseline* è un programma che analizza il pacchetto e lo rimanda fuori senza alcuna inferenza: è il pavimento contro cui si legge tutto il resto.

Tutte le colonne vengono dalla **stessa esecuzione**, sulla stessa macchina, nello stesso minuto. La riga "letture di tabella" è piena per la prima volta: fino alla riscrittura del capitolo precedente, per P2 e P3 diceva *n.d.*

10.2 Primo risultato: la dimensione non predice la velocità

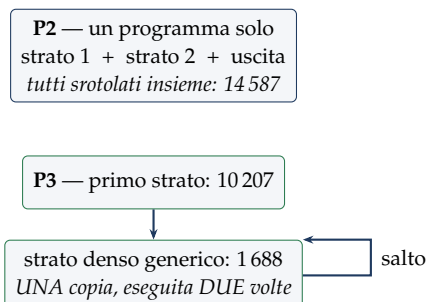
P3 ha il 18% di istruzioni in meno di P2 ed è il 61% più lento.

Il motivo sta nelle due righe centrali: tre salti fra programmi invece di uno, e ventinove letture di tabella per pacchetto invece di undici.

Il conteggio delle istruzioni misura quanto è *grande* il programma caricato, non quanto *lavoro* fa per pacchetto. In questo regime a dominare sono le letture di memoria e i salti, non l'aritmetica.

Perché P2 è più grande, pur essendo più veloce

Le due cose hanno la stessa causa: **P3 non tiene la rete intera in un programma.**



P3 srotola **un** solo strato denso generico e ci rientra, con un salto fra programmi, a ogni passo della rete. Così la *profondità* del modello non costa dimensione: un modello a quattro

strati gira sullo stesso identico binario. P2 deve srotolarli tutti, perché è un programma solo — e un modello più profondo richiede un altro binario.

Quel riuso è esattamente ciò che P3 paga in latenza: ogni rientro è un salto più le letture che servono a ricostruire il contesto perduto.

La stessa scelta di progetto spiega *entrambi* i numeri: P3 è più piccolo **perché** riusa un pezzo, ed è più lento **perché** riusarlo costa un salto e delle letture ogni volta.

10.3 Secondo risultato: quanto valgono le latenze assolute

Ogni cifra di latenza in tabella è il **minimo di sette misure** indipendenti. Guardando come sono distribuite quelle sette, dentro una singola esecuzione:

	baseline	P1	P2	P3
minimo	29 ns	73 ns	262 ns	421 ns
mediana	30 ns	88 ns	278 ns	453 ns
massimo	32 ns	99 ns	327 ns	476 ns
dispersione	10%	36%	25%	13%

Il 36% di P1 è *dentro* una sola esecuzione, sullo stesso binario, in pochi secondi. Le **istruzioni sono identiche** fra esecuzioni diverse: sono deterministiche. Le latenze no.

Su questa macchina virtuale una differenza di latenza sotto il 20%–30% **non significa nulla**, nemmeno dentro la stessa esecuzione. Valgono i rapporti fra le colonne, non le cifre.
Ogni valore assoluto va letto come ordine di grandezza su questa VM, mai come prestazione del sistema.

Un caso in cui la regola è servita

Il tetto sulle code di P3, riportato da 1 a 8, aggiunge **3 357 istruzioni** al programma:

	tetto 1	tetto 8
istruzioni (P3)	8 674	12 031
letture per pacchetto	29,0	29,0
latenza	419 ns	421 ns

Il 39% di istruzioni in più costa **2 nanosecondi**, cioè niente, e le letture non si muovono di una. Il motivo è semplice: il modello depositato non dichiara la feature delle code, quindi quel ramo del programma *non viene mai eseguito*. Cresce la dimensione statica; il percorso eseguito è lo stesso.

Una metrica che si sposta del 39% senza che cambi nulla di ciò che gira **non è una misura di costo**. Il numero di istruzioni va letto come "quanto è grande il binario", mai come "quanto lavora".

Questo ha reso necessario rifare la tabella dei risultati: la versione precedente presentava tre esecuzioni etichettate come ripetizioni, ma venivano in realtà da *stati diversi del codice*.

10.4 Terzo risultato: la genericità ha un costo di verifica

P2 serve qualsiasi modello che stia sotto certi tetti dichiarati a priori (numero massimo di neuroni, di ingressi, di feature). È il suo pregio: non ricompila mai.

Il conto però lo paga il verificatore del kernel, che percorre ogni cammino del corpo srotolato. Con i tetti larghi il kernel si arrende:

```
processed 1000001 insns (limit 1000000) ... total_states 13864
```

Da notare: il programma è lungo circa 14 900 istruzioni, **ampiamente dentro** il limite di dimensione. Quello che finisce è la *complessità di verifica*, che è un limite diverso e cresce col **prodotto** dei tetti:

$$\text{costo di verifica} \sim H_{\max} \times F_{\max} \times N_{\max}.$$

È la misura concreta di quanto un programma "generico" possa esserlo. La genericità di P2 non è gratis: si paga in stati del verificatore, e il tetto si tocca con numeri ragionevoli (8 neuroni, 4 feature, 128 ingressi).

10.5 Quarto risultato: togliere il compilatore dal nodo

P1 scrive i pesi *dentro* il codice come numeri letterali. È ciò che la rende veloce — il compilatore può far sparire le moltiplicazioni per zero e trasformare in scorrimenti quelle per potenze di due — ma ha un prezzo: ogni volta che il modello cambia, bisogna **rieseguire il compilatore sul nodo**.

Misurato in questa esecuzione: **1 242 ms**. Più di un secondo in cui quel nodo non sta aggiornando niente.

L'alternativa è compilare *prima*, su una macchina qualunque, e spedire al nodo l'oggetto già pronto. Al momento del deploy il nodo apre il file e lo carica, senza compilatore:

	con compilatore sul nodo	oggetto precompilato
costo di messa in servizio	1 242 ms	37 ms
istruzioni	1 026	979
latenza per pacchetto	73 ns	74 ns

Due ordini di grandezza sul costo di aggiornamento, e **nessun costo per pacchetto**: 74 ns contro 73 ns è la stessa cifra, dentro il rumore misurato sopra. La riduzione fatta dal compilatore sui pesi letterali resta dentro l'oggetto.

Una cautela sul numero 37: il costo di caricamento varia molto fra esecuzioni — sono stati osservati 4,7, 6,2, 20,7 e 37,2 ms per lo stesso oggetto. Quello che regge è l'*ordine di grandezza* rispetto al secondo e un quarto del compilatore, non la cifra precisa.

Vale solo quando i modelli sono noti in anticipo, che è l'ipotesi sotto cui P1 esiste.

10.6 Che cosa questi numeri non dicono

BPF_PROG_TEST_RUN esegue il programma in un ciclo sullo stesso buffer. Niente scheda di rete, niente driver, nessuna allocazione, nessuna pressione di cache da traffico vero.

Il valore in "milioni di pacchetti al secondo" è semplicemente $1/\text{latenza}$: un **picco teorico**, non un throughput retto. Per quello serve traffico vero, generato a ritmo, e la misura di quanti pacchetti si perdono.

Come cresce il costo

11.1 La domanda

Fin qui tutto è misurato su *un* modello e *una* topologia. La domanda che manca è come cambia il costo quando la rete o il modello diventano più grandi — e soprattutto se le tre pipeline hanno **pendenze diverse**.

Se le pendenze fossero uguali, averne più di una sarebbe un esercizio. Non lo sono, ed è questo il capitolo che lo mostra.

Le pipeline confrontate sono **quattro**, e formano una scala di *quanto si sa quando il programma viene compilato*:

	compilato dentro	un binario per
P1 specializzata	pesi e indice di questo nodo	nodo
P1.5 hardcoded	pesi; il nodo si legge a runtime	modello
P2 template	solo i soffitti	tutto
P3 modular	anche la profondità è a runtime	tutto

La P1.5 è quella che i capitoli precedenti chiamano semplicemente P1: la *disposizione* delle feature è scritta nel codice, ma l'indice del nodo no. La P1 specializzata congela anche quello — ed è il confronto con cui si chiude il capitolo.

11.2 Come è impostata la misura

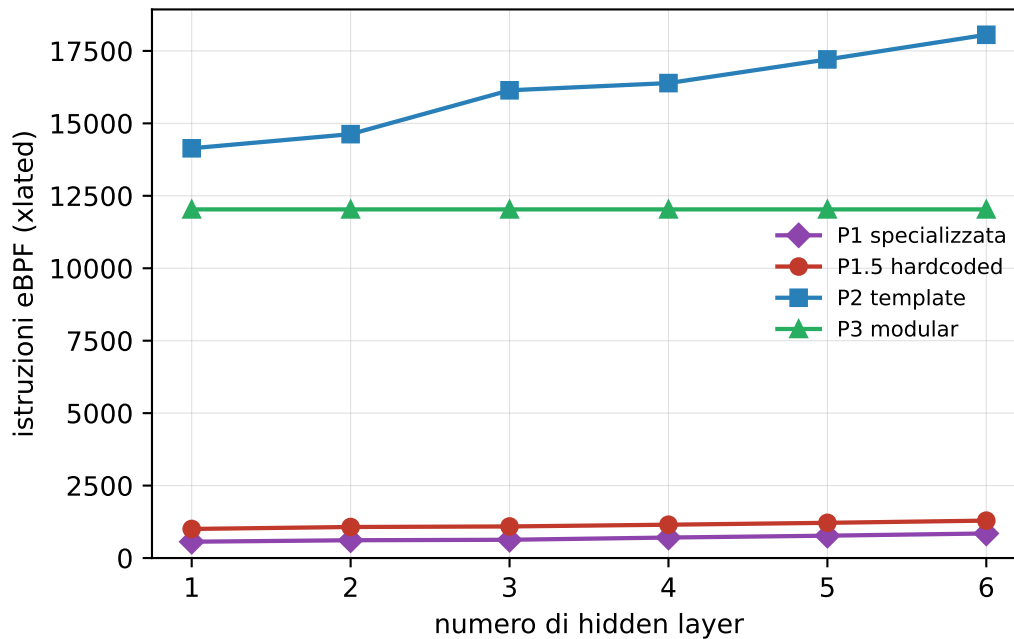
Cinque assi, e su ciascuno si muove **una sola** variabile: tutto il resto resta fermo, così una curva si può attribuire a quello che c'è scritto sull'asse delle ascisse e a nient'altro.

asse	valori	si ferma perché
nodi della rete	10, 25, 52, 75, 100	il tetto sugli ingressi è 128
hidden layer	da 1 a 6	—
neuroni per layer	2, 4, 6, 8	il tetto compilato è 8
composizione dell'IV	4 varianti	0, 1 o 2 one-hot
pesi a zero	0% ... 90%	—

Tre accortezze che hanno cambiato i risultati, non solo la forma:

- **Ogni misura in un processo separato.** P1 può far abortire il compilatore in modo non recuperabile; e un rifiuto del verificatore è un *dato*, non un guasto dello strumento.
- **I pesi vengono da un sacchetto fisso**, e un modello più grande *estende* quello più piccolo invece di sostituirne tutti i pesi. Senza questo, spostandosi lungo un asse cambiavano due cose insieme, e parte della curva di P1 era rumore dei pesi travestito da asse x.
- **Un allarme automatico sulle misure sporche:** se una pipeline ha lo stesso identico programma e la latenza varia più del doppio lungo l'asse, lo scarto è la macchina e lo strumento lo dice.

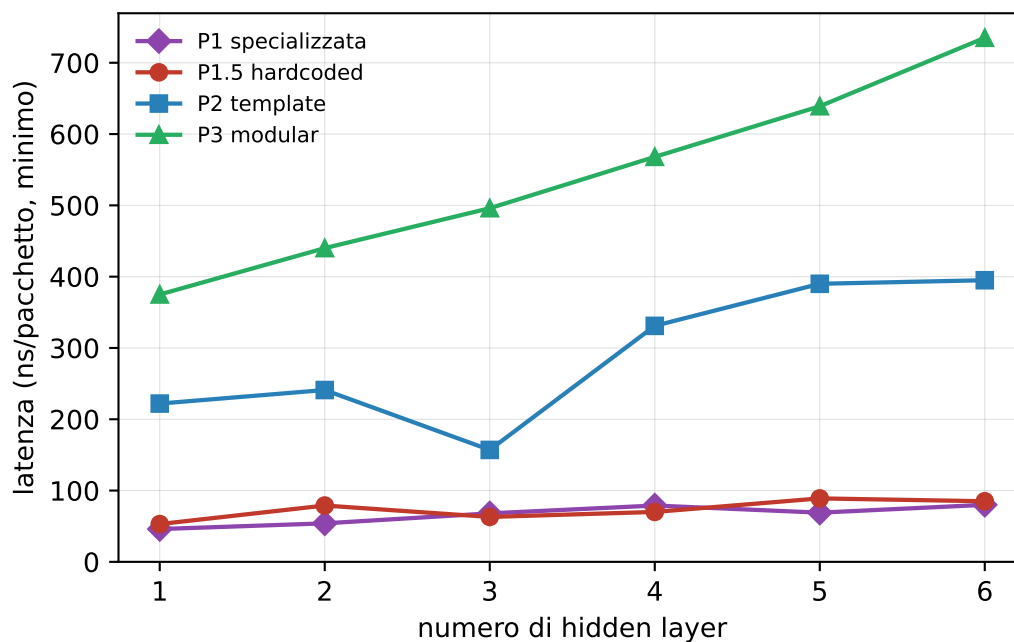
11.3 Profondità: chi paga in spazio e chi in tempo



P2 cresce di circa 780 istruzioni per ogni hidden layer. P3 di zero.

P3 srotola *un* solo strato denso generico e ci rientra a ogni passo della rete, quindi la profondità non entra nel programma: la sua riga è perfettamente orizzontale, da uno a sei strati. P2 deve srotolarli tutti, perché è un programma solo.

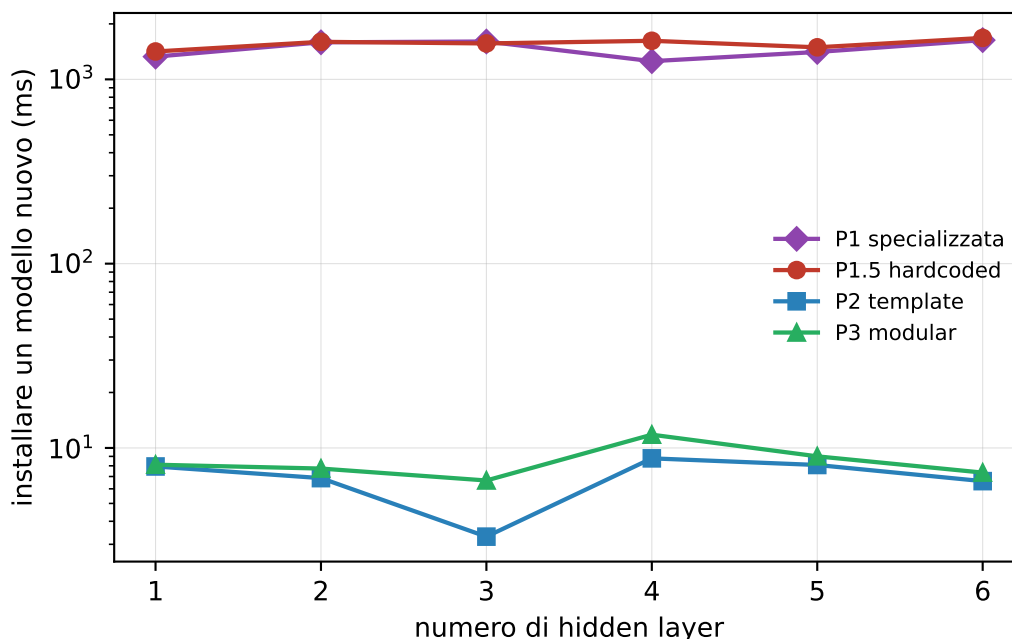
Ma la storia non finisce lì:



E P3 lo paga in tempo: circa 70 nanosecondi per strato, contro i 36 di P2. Sono i salti fra programmi, da 2 a 7.

Le due figure hanno la *stessa* causa vista dai due lati. P3 è più piccolo **perché** riusa un pezzo, ed è più lento **perché** riusarlo costa un salto e delle letture ogni volta. È il compromesso della modularità, ridotto a due numeri.

11.4 Aggiornare il modello: due ordini di grandezza



Attenzione alla scala verticale: è **logaritmica**. Su scala lineare le due curve basse sarebbero schiacciate sullo zero e si vedrebbe una curva sola.

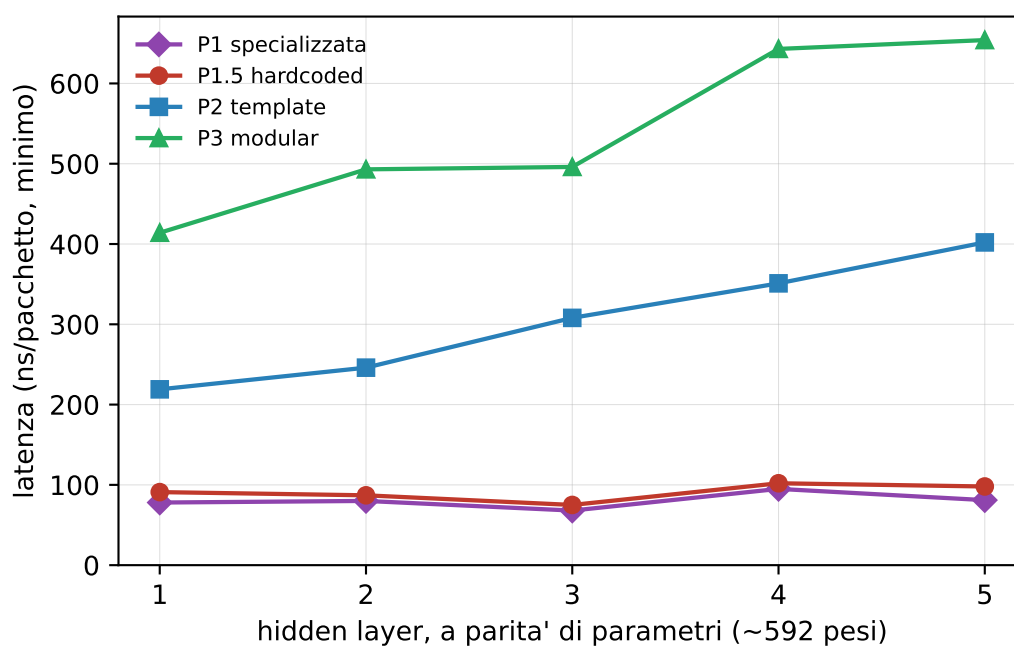
Installare un modello nuovo su un nodo già in servizio costa **circa 1,4 secondi** in P1 — che deve rigenerare il C e richiamare il compilatore — e **circa 7 millisecondi** in P2 e P3, che scrivono in una tabella.

È la misura che decide se una pipeline è usabile in una rete che cambia.

11.5 Larga o profonda? Dipende da chi la esegue

Dato un numero di parametri da spendere, conviene metterli in *un* layer largo o in *molte* stretti? È una domanda che si fa di solito parlando del modello, come se la risposta fosse una proprietà della rete neurale. Qui non lo è.

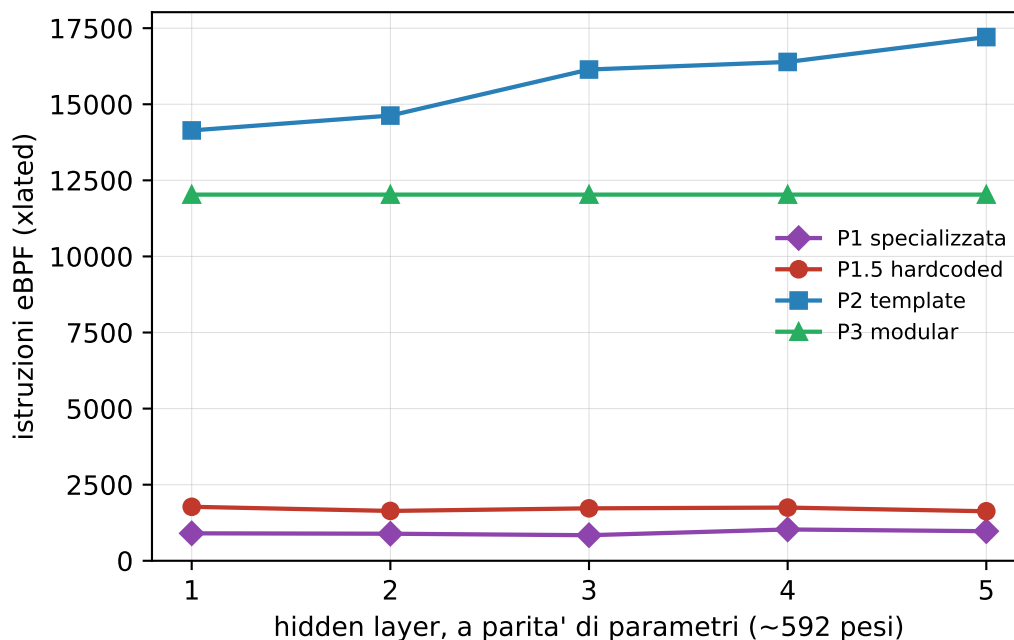
Cinque forme, tutte con lo stesso budget — 592 pesi, con uno scarto del 2% — disposto in modo diverso: da un solo strato da otto neuroni a cinque strati stretti. Tutte stanno dentro i soffitti compilati, quindi tutte e quattro le pipeline possono eseguirle tutte.



A parità di parametri, le due versioni hardcoded **non distinguono** una rete larga da una profonda. P2 perde l'84% di latenza e P3 il 58%.

La stessa scelta di architettura costa zero o costa il doppio, a seconda della pipeline su cui finisce.

E i due che perdono, perdono per ragioni diverse:



P2 cresce anche in dimensione (dal 14 140 al 17 207): deve srotolare ogni strato, perché è un programma solo. P3 ha la riga **perfettamente orizzontale** — dodicimila e trentuno istruzioni a tutti e cinque i punti — e perde comunque il 58% di velocità.

Dimensione costante e tempo crescente è la firma di un costo di transizione, non di calcolo. In P3 si vede allo stato puro: ciò che cresce non è il lavoro, è il numero di volte che il programma deve rientrare in sé stesso.

E la conclusione di prima?

Un capitolo fa avevamo scritto che allargare batte approfondire. Qui le due P1 dicono di no. Non è una contraddizione: **dipende dal budget**.

budget	larga	profonda (8 strati)
~300 pesi	54 ns	69 ns (+28%)
~1 200 pesi	124 ns	282 ns (+127%)

A 592 pesi e cinque strati l'effetto su P1 sta sotto il rumore della misura; a 1 200 pesi e otto strati è più che doppio.

E c'è una seconda condizione, emersa provando quattro vettori d'ingresso diversi a budget abbinato. Con un ingresso **grande e dominato da una one-hot** — il caso di IPA, 52 dei 65 valori — allargare batte approfondire nettamente: +140% di latenza a otto strati. Con un ingresso **piccolo e denso** (undici valori, nessuna one-hot) il vantaggio **sparisce**: a 300 pesi la versione a otto strati è perfino un poco più veloce, e più piccola.

Il motivo è lo stesso di due sezioni fa. Una one-hot larga costa poco per pacchetto, perché il datapath ne legge *una colonna sola*; un ingresso denso invece fa moltiplicare le letture per la larghezza del primo strato.

La frase giusta non è "le reti larghe sono preferibili a quelle profonde". È: *con un ingresso dominato da feature one-hot, allargare batte approfondire, e sempre di più al crescere del budget; con un ingresso piccolo e denso il vantaggio sparisce, e oltre una certa taglia è la forma larga a non compilare più.*

La prima è una legge, e i dati non la sostengono. La seconda è un risultato con un dominio di validità.

Il limite mangia prima le larghe

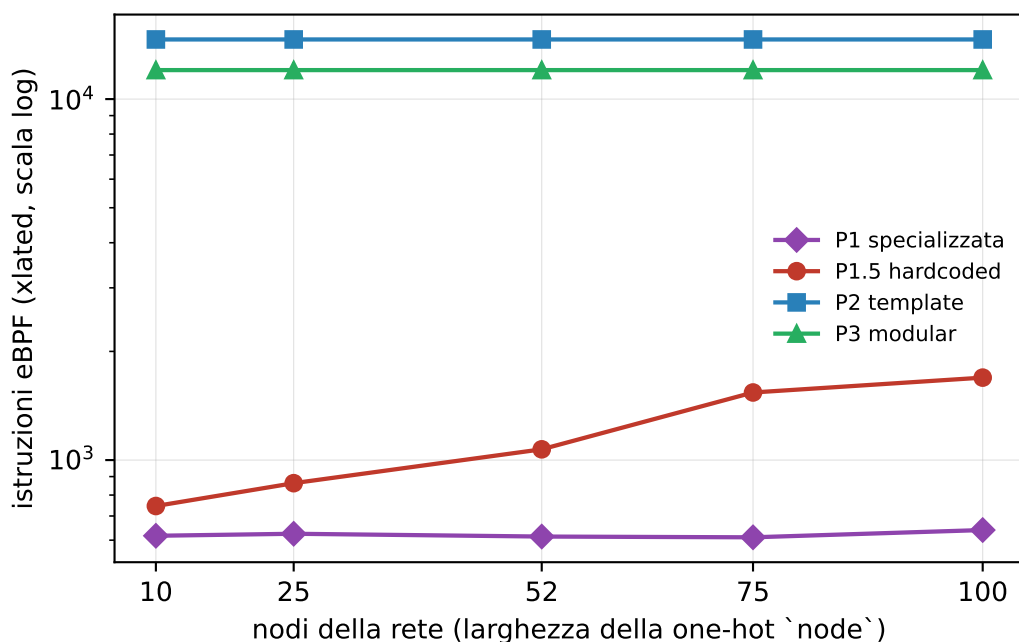
A parità di parametri la forma larga sfonda lo stack eBPF da 512 byte *prima* di quella profonda, perché è il primo strato a consumarlo. Su undici ingressi e 1 200 pesi, in ordine di stack stimato: larga 1×63 crash, 2×26 crash, profonda 4×17 carica, profonda 8×11 carica.

Con un ingresso piccolo e un budget medio, **la rete profonda è l'unica che sta in piedi**.

11.6 Congelare il nodo: conviene?

L'indice del nodo è una **costante del deployment**: un nodo non cambia di posto. Il codice però lo legge a runtime, e per farlo porta uno switch con un caso per ogni nodo della rete — 52 casi per Germany50, di cui ne esegue *uno*.

Congelandolo a tempo di generazione lo switch sparisce, e con lui la lettura della tabella. Restano quattro numeri costanti, che il compilatore può piegare dentro al conto.



La taglia della rete entra nel programma **solo** nella P1.5, che passa da 746 a 1692 istruzioni fra 10 e 100 nodi.

Congelando il nodo quella dipendenza **sparisce**: la riga viola è piatta, fra 611 e 640 a qualunque taglia.

(La scala è logaritmica: in lineare le due righe P1 resterebbero schiacciate contro le quattordicimila di P2, e il confronto fra loro — che è il punto della figura — non si vedrebbe.)

Ma la velocità non cambia

odi	10	25	52	75	100
P1 specializzata	58	68	54	55	55 ns
P1.5 hardcoded	61	60	57	59	61 ns

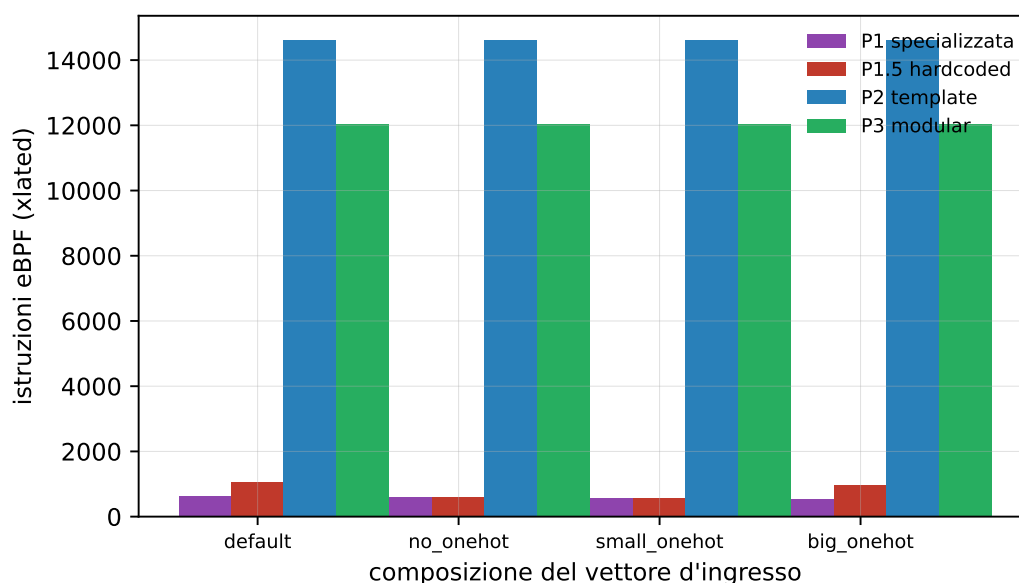
Praticamente identiche, dentro il rumore.

Il motivo per cui la *dimensione* crolla e il *tempo* no è lo stesso: lo switch ha 52 casi ma ne esegue uno. A runtime è un salto indicizzato, che costa poco; quelle 208 assegnazioni sono compilate per eseguirne quattro.

Toglierle libera molto **spazio** e quasi nessun **tempo**.

Il controllo, e perché ci si può fidare

Come si fa a essere sicuri che il divario venga davvero dalla one-hot del nodo e non da qualcos'altro che è cambiato per caso? Si prende un vettore d'ingresso che la feature *non* la contiene:



Nelle due composizioni senza la feature node le due P1 sono **identiche alla cifra** (599 e 599; 575 e 575). In quelle che ce l'hanno, divergono. Il divario è tutto lì e nient'altro.

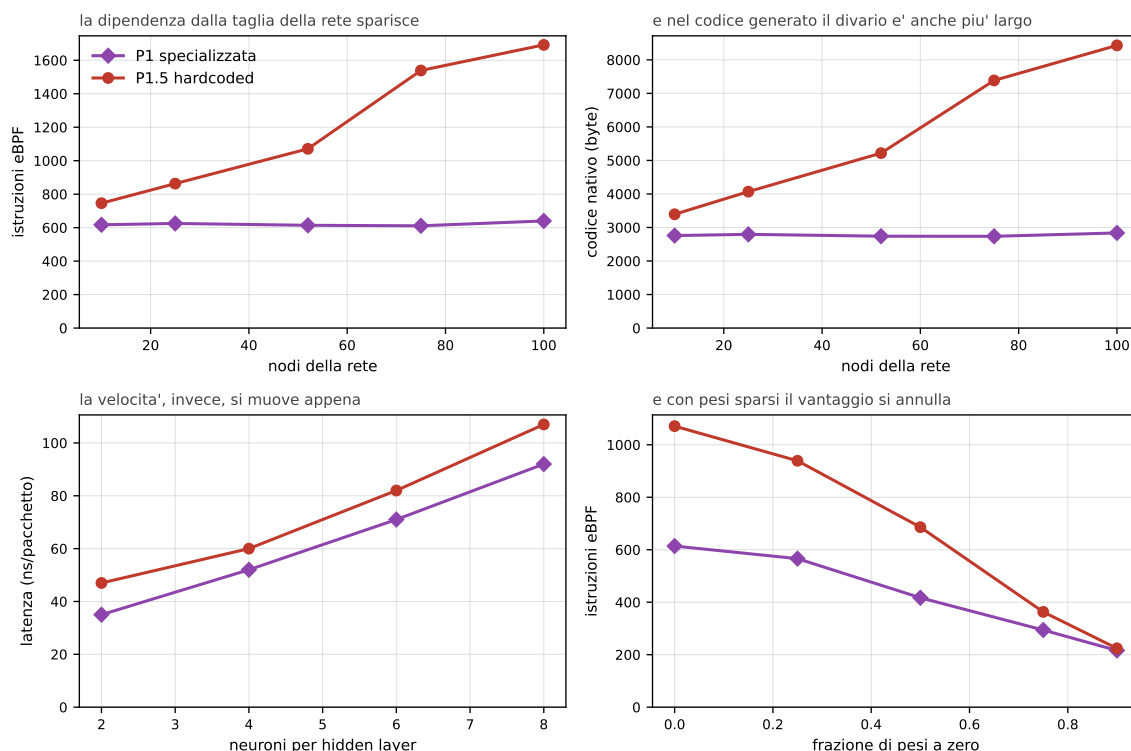
C'è poi una verifica che viene prima di ogni numero. Congelare il nodo significa scegliere *una colonna* della matrice dei pesi al momento di generare il codice: sceglierne una sbagliata darebbe un programma più piccolo, più veloce, e che calcola **un altro modello** — e nessuna misura di costo se ne accorgerebbe. Ottanta casi, dieci valori di TTL per otto configurazioni di link: la versione congelata decide sempre la stessa classe di quella che legge il nodo dalla tabella.

I due vantaggi non si sommano

pesi a zero	0%	25%	50%	75%	90%
P1 specializzata	614	566	417	294	216
P1.5 hardcoded	1 071	939	686	363	224

Al 90% di zeri il divario è sparito. Sparsità e nodo congelato sono **due strade alla stessa riduzione**: se i pesi sono già quasi tutti zero, il compilatore cancella lo switch da solo e non resta niente da congelare.

Le quattro risposte in una figura



Qui ci sono solo le due P1: in tutte le altre figure P2 e P3 stanno un ordine di grandezza più su e schiacciano questo confronto sul fondo del grafico.

I quattro pannelli non dicono la stessa cosa, ed è questo il punto. La dimensione crolla e smette di dipendere dalla rete; il codice nativo crolla ancora di più (2 837 byte contro 8 432 a cento nodi); la velocità si muove appena; e con i pesi sparsi le due curve si toccano.

Due aspettative smentite

su Germany50	specializzata	P1.5
istruzioni eBPF	614	1 071
codice nativo (byte)	2 740	5 218
latenza	54 ns	57 ns
memoria delle tabelle	2 356 B	2 356 B
aggiornare il modello	1 550 ms	1 460 ms

Le ultime due righe meritano attenzione perché smentiscono due aspettative ragionevoli.

La **memoria delle tabelle non cambia**: il programma specializzato non *legge* più la tabella del nodo, ma continua a *dichiararla*, perché l'intestazione è condivisa. Sono pochi byte, ma è codice morto.

Il **costo di aggiornamento non cala**: il sorgente è più piccolo, ma il tempo se lo mangia l'avvio del compilatore, non la lunghezza del file. La specializzazione non rende la compilazione più veloce — e siccome ne servono N invece di una, il costo totale di installazione *peggiora* di un fattore N.

Il verdetto

Congelare il nodo **non è un'ottimizzazione di velocità**. È ciò che rende la pipeline hardcoded *praticabile su una rete grande*.

Lo switch della P1.5 cresce col numero di nodi moltiplicato per la larghezza del primo strato. Su una topologia da 500 nodi con 8 neuroni sarebbero quattromila assegnazioni srotolate, e c'è una taglia oltre la quale quel programma non si carica più. La specializzata non ci arriva mai.

Il prezzo è **un binario per nodo**: 52 compilazioni e 52 installazioni, una per macchina. Il costo di aggiornamento si moltiplica proprio per la grandezza da cui la specializzazione ha appena liberato la dimensione.

Su Germany50, dove la differenza è fra 614 e 1 071 istruzioni a parità di velocità, è una scelta legittima in entrambe le direzioni. Su una rete dieci volte più grande non lo è più.

11.7 Congelare quali porte esistono

La sezione precedente congela *dove* il nodo si trova. Questa congela *quante interfacce ha*, che è una cosa diversa e si presta a un equivoco che vale la pena togliere di mezzo subito.

Il modello riserva sei colonne per `link_state` perché sei è il grado del nodo **più connesso** della rete. Ma su Germany50 i gradi vanno da 2 a 5, la media è 3,52, e **nessun nodo arriva a sei**. Su un nodo di grado 3 le tre colonne restanti non sono link caduti: dietro non c'è alcuna interfaccia. Valgono zero adesso, varranno zero sempre.

Non si congela il **valore** della feature, si congela la sua **struttura**.

Se un link sia su o giù resta letto a runtime, esattamente come prima. Quello che diventa noto alla compilazione è *quali posizioni possono esistere*.

Il generatore prende l'insieme delle porte presenti e non emette affatto le colonne che restano fuori. Il termine che sparisce è $\text{link_state}[i] \cdot w[j][i]$ con $\text{link_state}[i]$ nullo: la somma è fra interi a 64 bit, quindi togliere un addendo nullo non cambia un bit. Gli offset dei pesi non si spostano, l'ordine delle feature non cambia, n_{in} resta quello del modello.

Quanto costa una colonna che non serve

porte presenti	2	3	4	5	6
istruzioni eBPF	546	564	582	598	614
latenza	54	47	58	49	53 ns

Le istruzioni scendono in modo lineare e pulito: gli scarti sono +18, +18, +16, +16, cioè **17 istruzioni per porta**. Un nodo di grado 2 porta l'**11,1 %** di programma in meno di uno di grado 6.

La latenza non si muove. Non è nemmeno monotona, e l'escursione è del 23% su un valore intorno ai 50 ns.

Fra grado 6 e grado 2 spariscono **sedici** moltiplicazioni-accumulo, che a 3,2 GHz valgono all'incirca 5 ns. Il rumore di questo banco è il doppio. L'effetto c'è nel codice ed è sotto la risoluzione della misura: la riduzione di dimensione è dimostrata, il guadagno di tempo **no**.

Un'aspettativa smentita

Mi aspettavo il contrario, e vale la pena dire perché. Nel caso del nodo il tempo non cambia per un motivo chiaro: lo switch ha 52 casi ma ne esegue *uno*, quindi quelle 208 assegnazioni sono codice mai eseguito. `link_state` è invece un vettore **denso**: quelle moltiplicazioni vengono eseguite davvero, a ogni pacchetto. Sembrava dovesse seguirne un guadagno visibile.

Il ragionamento è corretto ma la conclusione no: sono sedici operazioni su circa 979 istruzioni per pacchetto. Il meccanismo è reale, la sua ampiezza è trascurabile.

Il controllo, che è servito davvero

Il confronto è fra due programmi che differiscono *solo* per questo: il nodo è congelato in entrambi. Ottanta casi — dieci valori di TTL per otto configurazioni di link realizzabili — danno la stessa classe in tutti.

Ma quel numero, da solo, non dimostra niente. Se quelle colonne non spostassero mai la decisione, «le due versioni concordano» sarebbe vero anche per una specializzazione **sbagliata**. Serve un controllo al contrario: accendere uno slot che sul nodo non esiste, e pretendere che a quel punto le due divergano.

Al primo tentativo quel controllo è rimasto **muto**. Non per un difetto del codice: una colonna di `link_state` contribuisce $1 \cdot w$ a un accumulatore che ne somma 65, e su quei pesi non ribaltava mai l'argmax. È una proprietà dei *pesi*, non della specializzazione. Cercando fra più semi del generatore di pesi il controllo morde — 29 divergenze — e solo a quel punto l'ottanta su ottanta diventa una prova.

Un test che passa anche quando la sua ipotesi è violata non sta verificando l'ipotesi. Il primo run era **inconcludente**, non positivo, e il banco adesso fallisce invece di annottarlo in giallo.

Il controllo sull'asse

Le altre tre pipeline non specializzano nulla, quindi lungo questo asse devono restare ferme — e lo sono: 1 071, 14 985 e 12 349 istruzioni su tutti e cinque i punti, con 319 pesi ovunque. L'asse di per sé non costa niente, quindi la pendenza della riga specializzata viene da lì e da nient'altro.

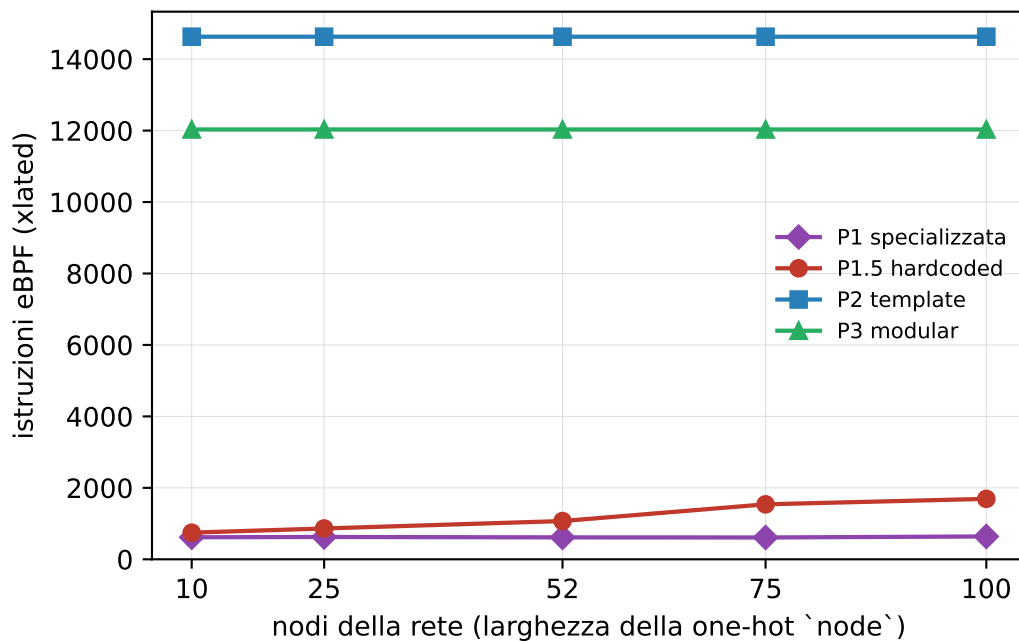
Vale anche il contrario: a sei porte il programma generato è **identico byte per byte** a quello non specializzato. I 457 punti di distacco che a Germany50 separano le due P1 sono quindi interamente la one-hot del nodo, e niente affatto le porte.

Quello che resta da misurare

L'incrocio con la sparsità. Con pesi quasi tutti nulli il compilatore cancella già per conto suo parte di quelle moltiplicazioni, e i due effetti potrebbero non sommarsi — è esattamente quello che succede fra nodo congelato e sparsità, dove al 90 % di zeri il divario sparisce.

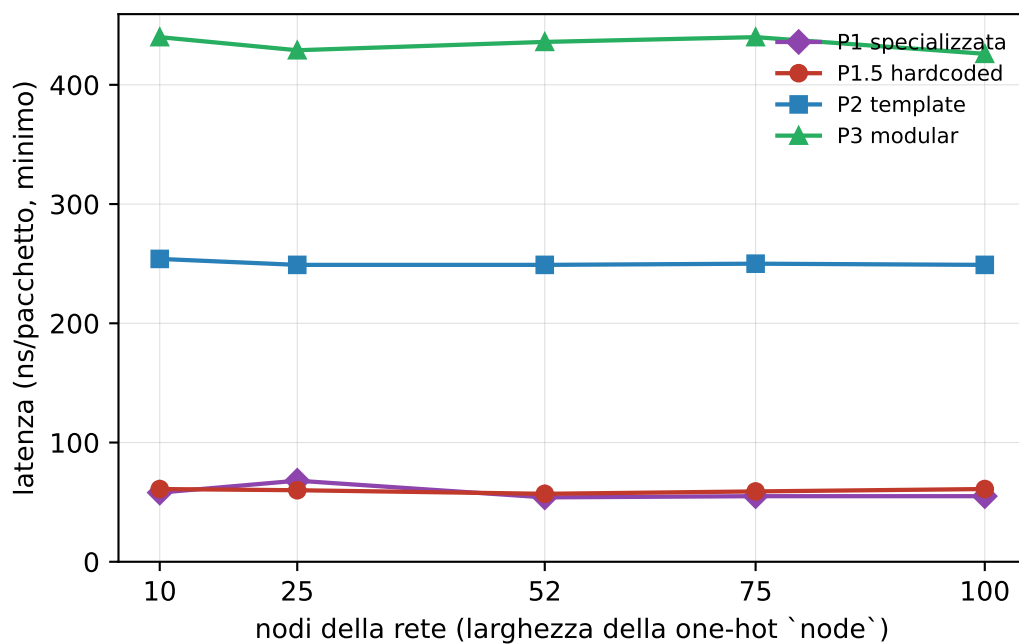
11.8 Due larghezze, due risposte opposte

Questa coppia di figure è il risultato che preferisco di tutta l'analisi, perché la risposta non è quella che verrebbe da indovinare.



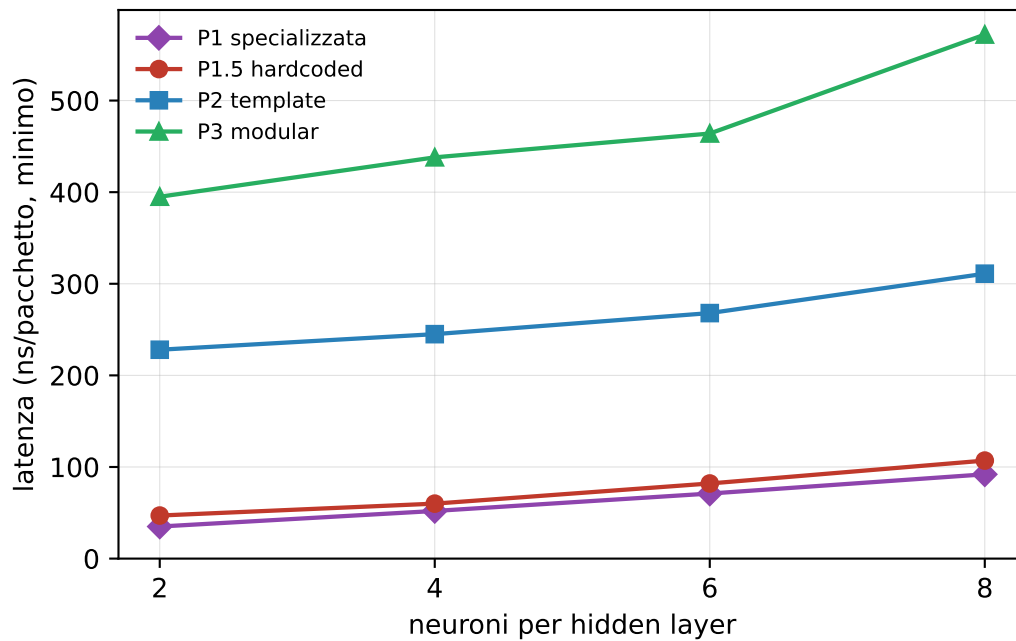
La taglia della *rete* entra nel programma solo in P1, che passa da 746 a 1 692 istruzioni fra 10 e 100 nodi: è lo switch della one-hot che si srotola. P2 e P3 restano alla cifra esatta a ogni punto, perché il loro codice non nomina mai la forma del modello.

E a runtime?



Piatte tutte e tre. Cento nodi costano quanto dieci. Il motivo è strutturale: una one-hot ha un solo uno, quindi il datapath legge *una sola colonna di pesi* qualunque sia la larghezza del vettore.

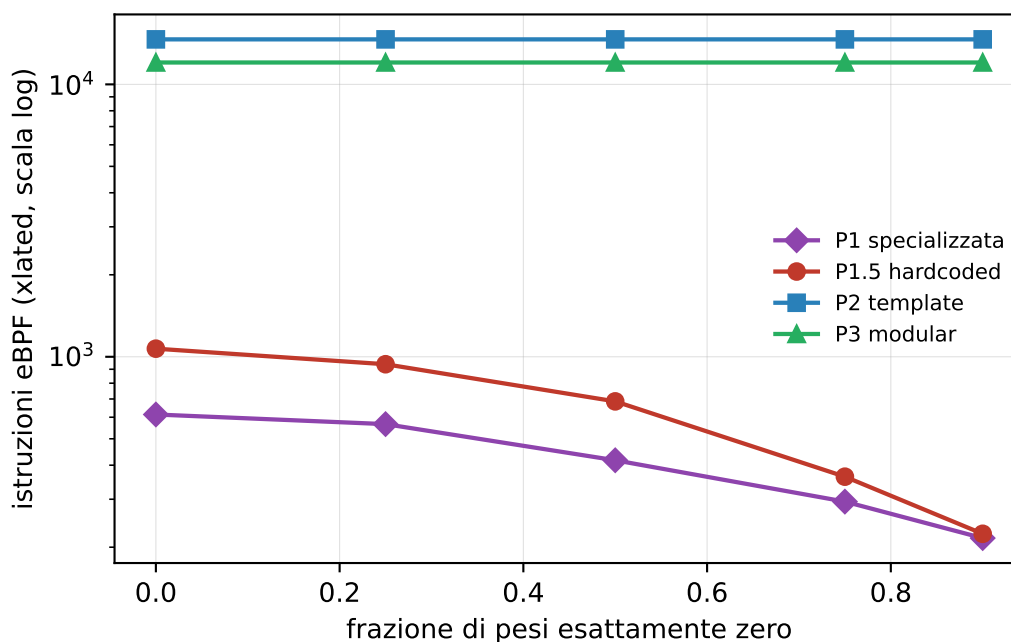
Allargare i layer *nascosti*, invece:



La rete può crescere quanto vuole; il modello no.

Allargare l'ingresso è gratis a runtime, perché una one-hot legge una colonna sola. Allargare i layer *nascosti* si paga, su tutte e tre le pipeline, perché ogni neurone in più sono pesi in più letti per ogni pacchetto.

11.9 I pesi contano, ma solo per una delle tre



Stessa identica forma in tutti e cinque i punti: cambia solo *quanti* pesi valgono zero.

Con il 90% dei pesi a zero, P1 passa da 1 071 a **224 istruzioni**. P2 e P3 non si muovono di una.

P1 scrive i pesi nel codice come numeri, quindi il compilatore vede una moltiplicazione per zero e la cancella. Per P2 e P3 uno zero è un byte in una tabella come un altro, e il programma è lo stesso.

Lo stesso vale per la composizione dell'ingresso: la figura del controllo, due sezioni più su, mostra che cambiare *quali* feature compongono il vettore d'ingresso ricompila entrambe le P1, mentre P2 e P3 leggono il descrittore da una tabella e non cambiano di una istruzione.

11.10 Il risultato che non cercavamo

Durante lo sweep una configurazione si è fatta rifiutare dal verificatore: quattro hidden layer da quattro neuroni. Le forme intorno caricavano tutte, inclusa quella *più grande* con cinque strati. Nessuna proprietà della profondità può produrre una cosa del genere.

Tenendo ferma la forma e cambiando solo il seme casuale dei pesi:

seme	esito
42	rifiutato
1	carica, 1 119 istruzioni
2	carica, 1 175
7	carica, 1 118
999	carica, 1 075

Il seme 1 carica a 1 119 istruzioni; il 42 viene rifiutato a 1 120. Una istruzione di differenza.

In P1, riaddestrare un modello senza toccare l'architettura può renderlo non caricabile. Stessa rete, stessi iperparametri, pesi nuovi, e il nodo lo rifiuta. P2 e P3 non hanno questo rischio: per loro i pesi sono byte in una tabella, e che il programma si carichi dipende solo dalla forma.

È il rovescio esatto del vantaggio di due pagine fa. La stessa riduzione del compilatore che porta P1 a 224 istruzioni è quella che lo rende imprevedibile. Un compromesso vero, misurato su entrambi i lati.

Il meccanismo preciso — *quale* limite del kernel — non è stato confermato, e non va inventato: il messaggio d'errore parla di un programma troppo lungo e cita un limite di 4096 istruzioni che nella stessa esecuzione P2 supera di quattro volte. È la stessa costante morta già incontrata sul capitolo del verificatore.

11.11 Che cosa questi grafici non dicono

- **Non si confrontano latenze prese da assi diversi.** A programma identico, P2 misura circa 250 ns su due assi e circa 310 sugli altri due: la macchina si è caricata a metà esecuzione. Dentro un asse i confronti reggono. Le istruzioni, che sono deterministiche, si confrontano ovunque.
- La figura sulla larghezza va letta per la *direzione*, non per la pendenza, per lo stesso motivo.
- Oltre circa 115 nodi non si va senza alzare un tetto compilato e **rimisurare**.

Il tempo fra arrivo e partenza

12.1 La domanda

Fin qui ogni numero di velocità di questo quaderno viene dalla stessa misura: si chiede al kernel di eseguire il programma centomila volte su un pacchetto che sta in memoria, e si divide il tempo. È una misura onesta e ripetibile, ma risponde a una domanda sola: *quanto dura il programma*.

La domanda che restava aperta è un'altra: se un generatore manda pacchetti veri su un collegamento vero, e dall'altra parte c'è il nodo con la pipeline attaccata, **quanti pacchetti al secondo regge**, quanti se ne perdono, e quanto tempo passa fra quando il pacchetto arriva e quando riparte.

Quest'ultima è la grandezza che conta davvero. Non è la durata del programma: è la durata del programma *più* tutto quello che sta intorno.

12.2 Due banchi che misurano due cose diverse

Questa distinzione ha creato più confusione di qualunque altra cosa in questo lavoro, quindi vale la pena spiegarla senza tecnicismi.

L'analogia

`test_suite` cronometra un atleta sul tapis roulant, in laboratorio. Ottieni la sua velocità pura, misurata benissimo, sempre uguale.

`bench_throughput` lo cronometra su una strada vera, con il traffico. Scopri quanto ci mette davvero ad arrivare — ma una parte di quel tempo è la strada, non l'atleta.

Servono **tutte e due**. La prima dice quale pipeline è più veloce. La seconda dice se il sistema regge. Nessuna delle due risponde alla domanda dell'altra.

In concreto:

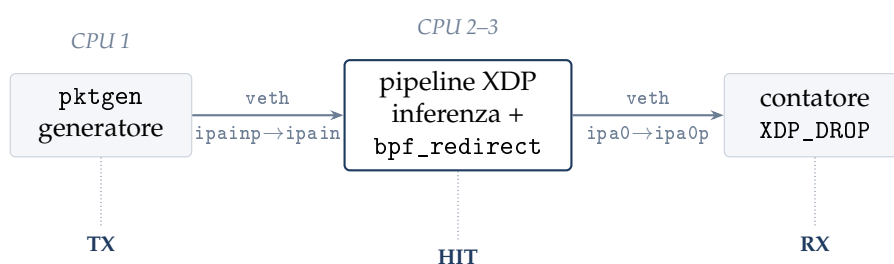
	test_suite	bench_throughput
Chi fabbrica i pacchetti	nessuno: ce n'è uno solo, in memoria, riusato	un generatore di traffico (<code>pktgen</code>), pacchetti veri
Dove gira il programma	dentro una chiamata di sistema	sull'interfaccia, dove girerebbe davvero
Il pacchetto viene consegnato?	no, non esce mai	sì, attraversa il collegamento ed esce dall'altra parte
Che cosa finisce nella cifra	solo il programma	programma + driver + code + schedulatore
Cifra tipica (baseline)	29 ns, cioè 34,5 Mpps	1 142 ns, 0,88 Mpps misurati
Serve per	confrontare le pipeline fra loro	dire se il sistema sta in piedi sotto carico
Riproducibile?	sì, è deterministico	la latenza sì; la portata solo come <i> differenza </i> rispetto alla baseline (§12.10)

I 34,5 Mpps della baseline non sono un throughput: sono il reciproco della durata di una funzione. Un pacchetto che non viene mai consegnato non ha un throughput.

Il rapporto fra le due colonne — 29 ns contro 1 142 — non è un difetto del secondo banco. È la risposta alla domanda *quanto costa la rete intorno al programma*, e su questa macchina la risposta è: quasi quaranta volte il programma stesso. Il conto, e il motivo per cui il confronto fra pipeline sopravvive lo stesso, stanno in §12.10.

12.3 L'architettura del banco, e che cosa si misura dove

Prima dei numeri conviene avere in testa il disegno, perché ogni cifra di questo capitolo è presa in un punto preciso di questa catena.



Tre soggetti e tre punti di conteggio. I soggetti:

- **Il generatore (TG)** è `pktgen`, che vive *dentro* il kernel: non è un programma che apre una socket, è un thread del kernel che fabbrica pacchetti e li consegna direttamente al dispositivo. È il motivo per cui arriva a centinaia di migliaia di pacchetti al secondo da un core solo, dove un mittente in spazio utente si fermerebbe molto prima.
- **Il nodo sotto prova (DUT)** è la pipeline eBPF agganciata a XDP sull'interfaccia d'ingresso. Fa l'inferenza, sceglie la classe, e chiama `bpf_redirect` verso l'interfaccia d'uscita corrispondente.
- **Il destinatario** è un secondo programma XDP, minuscolo, che incrementa un contatore e restituisce `XDP_DROP`. Conta senza consegnare, così il numero misura la consegna del datapath e non il costo di passare il pacchetto allo stack di rete.

I tre punti di conteggio, e perché non basta uno solo:

Punto	Chi lo conta	Che cosa dice
TX	<code>pktgen</code>	quanti pacchetti sono stati <i>accettati</i> dal dispositivo. I rifiutati a coda piena finiscono in un contatore separato (<code>errors</code>) e sono carico offerto, non consegnato.
HIT	la pipeline	quanti pacchetti il programma ha davvero elaborato. TX – HIT sono quelli entrati e mai arrivati all'inferenza.
RX	il contatore	quanti sono arrivati a destinazione. HIT – RX sono quelli elaborati e non usciti: <code>redirect</code> fallito, oppure la classe scelta era <code>DROP</code> .

Un solo numero di “perdita” confonderebbe tre cause che chiedono tre rimedi diversi. In tutte le misure di questo capitolo **HIT – RX vale zero e TX = HIT**: la pipeline non ha

mai perso un pacchetto che le sia stato consegnato. Tutto ciò che manca all'appello è stato *rifiutato all'ingresso* dal veth, cioè è contospinta — la prova che il nodo è saturo, non che sbagli.

Dove finisce il tempo, e dove si misura

La latenza è marcata **dentro** la pipeline: un timestamp quando il pacchetto entra nel programma, uno quando il programma d'uscita lo vede. Copre quindi inferenza *più* il tragitto fino all'altra interfaccia, non il tempo che il pacchetto passa in coda prima di essere elaborato.

La portata, invece, si misura **agli estremi**: pacchetti contati all'uscita, diviso la durata della finestra. Dentro quel numero c'è tutto, compreso ciò che non è la pipeline.

12.4 Il banco

Generatore e nodo sotto prova stanno sulla stessa macchina virtuale, collegati da una coppia veth. Non è l'ideale — su un banco vero sarebbero due macchine e un cavo — ma è quello che c'è, e le sue conseguenze sono note e dichiarate:

- veth rifiuta `clone_skb` e `burst`, le due manopole con cui `pktgen` normalmente riusa il buffer: consegna l'`skb` alla ricezione del peer, dove XDP lo riscrive, quindi non può dividerlo. Si paga un'allocatione per pacchetto.
- XDP pretende 256 byte di spazio davanti al pacchetto (`XDP_PACKET_HEADROOM`); gli `skb` di `pktgen` ne hanno 64 (`NET_SKB_PAD`). Quindi il kernel **copia ogni pacchetto** prima di eseguire il programma.
- Senza precauzioni, generatore e nodo finiscono sullo stesso core e si contendono la CPU: si misura lo schedatore. La NAPI in modalità thread — si accende scrivendo 1 in `/sys/class/net/<dev>/threaded` — sposta la ricezione in un thread del kernel che si può fissare su un core diverso. È l'unico modo, su questa macchina, per separare le due metà del banco.

Una trappola che è costata tempo: la NAPI in modalità thread va abilitata *dopo* aver attaccato il primo programma XDP. veth usa la NAPI solo quando un programma è attaccato, quindi prima non c'è niente da mettere in un thread, e la richiesta viene rifiutata con `Operation not supported`. Per lo stesso motivo fra una pipeline e l'altra il programma si **sostituisce**, non si stacca: staccarlo smonterebbe la NAPI e con lei la modalità thread.

12.5 Tre cose che rendevano il confronto ingiusto

La prima versione del banco misurava una pipeline alla volta, da capo. Il risultato è stato una riga in cui la **baseline** — che non fa inferenza, cioè fa strettamente meno lavoro di tutte le altre — risultava il 26% più lenta di una pipeline che ne fa di più. Un risultato del genere non è un dato: è la prova che il banco stava misurando altro.

Le cause erano tre, e ognuna bastava da sola:

1. **Ogni metodo ricostruiva il fabric.** Interfacce nuove, indici nuovi, e lo stato del generatore che non sopravvive al giro.
2. **Ogni metodo chiamava il compilatore subito prima di misurare.** Le pipeline generiche bruciano secondi di CPU che la baseline non brucia, e la misura partiva su una macchina in stati diversi.

3. **I metodi giravano in sequenza, una volta ciascuno.** Qualunque deriva della macchina — frequenza, pagine, un processo che si sveglia — si mappava sull'*ordine* dei metodi, premiando chi capitava per primo.

La correzione, nella stessa struttura:

1. **Un fabric solo**, costruito una volta e usato da tutti.
2. **Prima si compila e si carica tutto**, poi si misura. Durante le misure nessun compilatore gira.
3. **Si misura a giri**: in ogni giro tutti i metodi, e di ogni metodo si tiene la mediana fra i giri. Una deriva colpisce allora tutti allo stesso modo.

E poi un controllo che il banco fa su sé stesso, perché una misura che si autocertifica vale più di una che va interpretata:

Il controllo di validità

La baseline deve risultare la più veloce, entro il 10%. Non è un'ipotesi da verificare: è una **condizione logica**. La baseline esegue un sottoinsieme stretto del lavoro di ogni altra pipeline, quindi non può consegnare meno. Se il banco dice il contrario, il banco ha torto, non la pipeline — e il run va buttato.

La tolleranza del 10% c'è perché due pipeline entrambe limitate dal generatore non sono distinguibili, e un'inversione dentro il rumore non è contaminazione: è l'informazione che il throughput non le separa.

12.6 Misurare arrivo → partenza

Il tempo che interessa è quello fra l'ingresso del pacchetto e il momento in cui riparte. Si prende con due marcature temporali dentro il programma, prima e dopo il blocco che decide, e si accumula la differenza in un istogramma.

Due dettagli non ovvi, entrambi scoperti sbagliando:

- **Il contatore deve stare nello stesso oggetto BPF della pipeline.** Due oggetti separati non possono condividere una mappa senza appoggiarla al filesystem BPF. Quindi lo strumento viene *iniettato nel sorgente* della pipeline prima di compilarla, con le dichiarazioni inserite prima del blocco che le usa — al primo tentativo erano in fondo, e non compilava.
- **L'istogramma deve essere logaritmico.** Il primo tentativo aveva 256 celle da 16 ns, scelte per distinguere 228 da 243 da 267 ns: le differenze fra pipeline sono di quell'ordine. Sbagliato per la ragione opposta — quella finestra copre 4 μ s, e sotto carico la coda sta molto più in alto, quindi i percentili venivano riportati come " $> 4 \mu$ s", cioè non riportati. Con `bpf_log21` e 40 celle si copre tutto; la distinzione fine fra pipeline sta nel **minimo**, che è esatto e non passa dall'istogramma.

12.7 Il risultato: la latenza

Cinque pipeline, tre giri, frame da 64 byte, NAPI in thread attiva, controllo di validità superato.

Pipeline	minimo (ns)	sopra baseline	fra i tre giri
baseline	202	—	202 / 206 / 202
p1_static	234	+32	234 / 236 / 234
hardcoded	256	+54	254 / 259 / 256
template	400	+198	404 / 400 / 399
modular	492	+290	487 / 493 / 492

L'ordine è quello atteso e la stabilità è la cosa notevole: **pochi nanosecondi di variazione** su misure ripetute a distanza di minuti. Con generatore e nodo su core separati, la differenza è attribuibile alla pipeline e non alla macchina.

Si legge anche il costo della genericità in nanosecondi, non più solo in istruzioni: **leggere il descrittore a runtime costa circa 200 ns per pacchetto**, e passare ai salti fra programmi ne costa altri 90.

12.8 Il risultato che non regge: la portata

La stessa tabella riporta anche i pacchetti al secondo, e quei numeri **non sono un risultato**. La dispersione fra i tre giri, a pieno rate:

Pipeline	giro 1	giro 2	giro 3	dispersione
p1_static	2,18 M	2,07 M	2,04 M	±7%
modular	1,24 M	1,42 M	1,21 M	±17%
baseline	1,71 M	2,10 M	2,47 M	±44%
template	1,60 M	1,65 M	1,02 M	±62%
hardcoded	2,78 M	1,17 M	1,46 M	±137%

La conseguenza si vede a occhio nel riepilogo: la mediana di hardcoded (1,46 M) finisce *sotto* quella di template (1,60 M), quando hardcoded fa molto meno lavoro. Non è un'inversione: è la mediana di tre misure che variano di 2,4 volte.

Il banco adesso lo dice da solo. Ogni riga porta la dispersione fra i giri, e oltre il 25% la riga viene marcata e il controllo di validità fallisce, con un messaggio che distingue le due metà: *la portata di queste righe non è un risultato; la latenza delle stesse righe è un'altra misura, con un'altra dispersione*.

Sullo stesso identico run, la latenza minima di una pipeline varia dell'1% fra i giri e la sua portata del 137%. Sono due misure con due affidabilità diverse, e vanno riportate come tali — non lasciate in una tabella che il lettore prende per buona tutta insieme.

12.9 Un tentativo che non ha funzionato

Il conto che ha fatto partire il tentativo è questo: a circa 550 kpps per core il costo per pacchetto è ≈ 1800 ns, e l'inferenza ne spiega 60. Tutto il resto non è il datapath: è il *modo* in cui i pacchetti vengono fabbricati — un'allocazione per pacchetto più una copia per l'headroom.

Esiste una tecnica nota per togliere entrambi: BPF_PROG_TEST_RUN in modalità *live frames* (BPF_F_TEST_XDP_LIVE_FRAMES, kernel ≥ 5.18). Il kernel prende i frame da una *page_pool*, esegue il programma, e se il verdetto è XDP_REDIRECT li **trasmette davvero**. Niente skb, niente copia. È quello che usa xdp-trafficgen per generare a doppia cifra di Mpps da un core solo.

Implementato e misurato, su kernel 7.0. Il risultato:

	1 thread	4 thread
frame chiesti	2 000 000	2 000 000
scartati dal redirect	813 189	1 988 176
partiti	1 186 811	11 824
arrivati	1 186 816	3 184
consegnati	1,44 Mpps	0,05 Mpps

Contro i **2,10 Mpps** che lo stesso banco consegna con `pktgen`. Il generatore “veloce” consegna **meno** di quello lento, e con quattro thread crolla di trenta volte.

La spiegazione sta nella colonna degli scartati. I frame non muoiono nel nodo: muoiono al `redirect`, perché la coda della `veth` è piena. Il generatore in modalità live non ha alcun freno — infila frame in un anello a ritmo di `syscall`, e nessuno gli dice di rallentare — mentre dall'altra parte un solo thread `NAPI` li drena. Con quattro thread che spingono nello stesso anello, il 99,4% muore prima di partire.

Il collo di bottiglia non era l'allocazione degli `skb`: era il lato *ricezione* della `veth` su un core solo, e nessun generatore lo sposta. Rendere il produttore più veloce di un consumatore che non può accelerare non produce throughput: produce scarti.

Il codice è stato rimosso. Quello che resta è il numero: **su questo banco il tetto è $\approx 1,4$ –2,1 Mpps, ed è una proprietà del collegamento virtuale, non di IPA**. È anche la ragione per cui i 34,5 Mpps stimati dalla durata del programma non saranno mai misurabili qui: mancherebbe un tetto di oltre quindici volte, e il tetto non è il programma.

La cosa utile venuta fuori per sbaglio

Il generatore a frame XDP costruisce il pacchetto byte per byte, mentre `pktgen` scrive la propria intestazione sopra il primo byte di payload — che nel formato IPA è `model_id`, da cui il valore 190 che compare nei registri. Sono quindi due sorgenti di traffico **completamente diverse**: pacchetti diversi, cammino diverso nel kernel, allocatore diverso.

La latenza minima della baseline misurata con le due:

<code>pktgen, skb, copia per headroom</code>	202 ns
<code>frame XDP da page_pool, nessuna copia</code>	201 ns

Un nanosecondo di differenza. La misura di latenza **non dipende da come sono fatti i pacchetti**, e questa è la conferma più forte che abbiamo che la tabella della latenza misura la pipeline e non il banco. Un esperimento fallito ha prodotto il controllo che all'esperimento riuscito mancava.

12.10 Le stesse pipeline, quaranta volte più lente

Fin qui il quaderno ha riportato due famiglie di numeri senza mai metterle nella stessa unità di misura, e questo ha nascosto la cosa più interessante. Adesso sono entrambi in **milioni di pacchetti al secondo**.

A sinistra `test_suite`, che esegue il programma centomila volte sullo stesso pacchetto in memoria e divide il tempo. A destra il banco con `pktgen`: traffico vero, frame da 512 byte, perdita 0,00%, generatore e nodo sullo stesso core (modalità *softirq*), controllo di validità superato.

Pipeline	test_suite	banco	rapporto
baseline	34,48 Mpps	0,876 Mpps	39,4×
P1 hardcoded	13,70 Mpps	0,810 Mpps	16,9×
P2 template	3,82 Mpps	0,655 Mpps	5,8×
P3 modular	2,38 Mpps	0,574 Mpps	4,1×

Quaranta volte, per la baseline. La prima reazione è che una delle due misure sia sbagliata. Non lo è nessuna delle due: misurano due cose diverse, e il modo per vederlo è girare la tabella in nanosecondi per pacchetto.

Pipeline	test_suite	banco	differenza
baseline	29 ns	1 142 ns	+1 113 ns
P1 hardcoded	73 ns	1 235 ns	+1 162 ns
P2 template	262 ns	1 526 ns	+1 264 ns
P3 modular	421 ns	1 743 ns	+1 322 ns

La differenza non è un fattore: è un **addendo**, e vale fra 1 113 e 1 322 nanosecondi — varia del 19% mentre le pipeline variano di quattordici volte. Non è rumore e non è un difetto della misura: è il costo di tutto ciò che sta *intorno* al programma, ed è quasi lo stesso per chiunque lo attraversi.

Quel migliaio di nanosecondi ha tre voci identificabili, tutte già incontrate:

1. **Un'allocazione per pacchetto.** Su veth il kernel rifiuta `clone_skb`: il buffer non si può riusare, perché viene consegnato al peer, dove XDP lo riscrive. `test_suite` riusa sempre lo stesso.
2. **Una copia per l'headroom.** XDP pretende 256 byte davanti al pacchetto, gli `skb` di `pktgen` ne riservano 64: il kernel copia ogni pacchetto prima di eseguire il programma.
3. **Il generatore stesso.** In modalità *softirq* la ricezione gira sulla CPU che ha trasmesso, quindi il tempo per pacchetto è *generazione più elaborazione*, non elaborazione soltanto.

Nessuna delle tre è il datapath. Tutte e tre sparirebbero con due macchine e una scheda di rete vera — che è esattamente il motivo per cui le cifre assolute di questo banco non vanno citate come prestazioni del sistema.

12.11 Perché il confronto regge lo stesso

Se l'addendo è comune, allora *sottrarlo* restituisce il costo della sola inferenza. È il conto che rende utile un banco altrimenti saturo:

Sopra la baseline	test_suite	banco	rapporto
P1 hardcoded	+44 ns	+93 ns	2,1×
P2 template	+233 ns	+385 ns	1,7×
P3 modular	+392 ns	+601 ns	1,5×

L'ordine è identico, le distanze sono dello stesso ordine di grandezza, e i rapporti stanno fra 1,5 e 2,1 — cioè sotto traffico vero l'inferenza costa circa il doppio di quanto costi in laboratorio. È un risultato in sé, e ha una spiegazione: il ciclo caldo di `test_suite` tiene pesi e codice in cache e ammortizza i salti fra programmi, mentre sotto traffico continuo

ogni pacchetto porta con sé pressione di cache. Le pipeline che usano salti e letture di mappa — P2 e P3 — sono quelle che pagano di più questa differenza, ed è coerente con il fatto che il rapporto *scende* al crescere del lavoro: chi fa già molto lavoro utile ammortizza meglio l'overhead fisso.

Che cosa vuol dire “saturato” qui

In questo banco il collo di bottiglia non è mai la pipeline. In modalità *softirq* il tetto è il generatore, che condivide il core; a core separati il tetto è la coda del *veth*, fissa a 256 descrittori. In entrambi i casi il nodo sotto prova **non arriva a saturare**, e infatti la perdita osservata è zero.

Questo toglie un risultato e ne lascia un altro. Toglie la *portata massima*: il numero in Mpps è il tetto del banco, non della pipeline, e va riportato come limite inferiore. Lascia il *costo per pacchetto*: siccome non ci sono code di mezzo, 1/pps è il tempo del percorso completo, e la baseline percorre la stessa strada meno l'inferenza. La sottrazione elimina il banco e lascia la pipeline.

Le percentuali dicono la stessa cosa da un'altra angolazione. Rispetto alla baseline, P3 vale il 6,9% sotto *test_suite* e il 65,5% sul banco: l'overhead comune **comprime** le differenze, e le comprime tanto più quanto è grande rispetto al lavoro utile. Per questo un banco così non serve a dire “P3 è quindici volte più lenta della baseline” — quella cifra è di *test_suite* — ma serve a dire che l'ordine e le distanze in nanosecondi si ritrovano anche quando i pacchetti sono veri. Le due misure non si contraddicono: una isola il programma, l'altra lo pesa dentro un sistema.

Un banco saturo non è un banco inutile: è un banco in cui vanno citate le *differenze* e non i *valori*. La regola pratica che ne è uscita: se il collo di bottiglia dichiarato è il generatore o il trasporto, la colonna in Mpps si riporta con il “maggiore o uguale” e il confronto si fa sulla colonna in nanosecondi, dopo aver sottratto la baseline.

12.12 Che cosa questi numeri non dicono

- **Non sono prestazioni di sistema.** Generatore e nodo condividono memoria, cache e schedulatore. Su hardware vero le cifre assolute sarebbero altre; i *rapporti* fra pipeline sono la parte che sopravvive.
- **La portata non separa le pipeline veloci.** Baseline, *p1_static* e *hardcoded* sono limitate dal generatore, non da sé stesse. Fra loro il throughput non dice niente: la latenza sì.
- **Il secondo salto conta.** Ogni pacchetto rediretto attraversa un'altra *veth* e un altro programma XDP — il contatore in uscita. Su un nodo vero il redirect andrebbe su una scheda di rete. Quel costo sta dentro la cifra ed è dichiarato, non sottratto.
- **Una sola taglia di frame è stata misurata a fondo** (64 byte). Lo sweep su 512 e 1514 è impostato ma non ancora eseguito con il banco equo.

Quel che resta

13.1 Fatto

- semantica delle classi dichiarata, mai dedotta; catena classe → azione → porta logica → interfaccia;
- motore indipendente dallo scenario, dimostrato su cinque topologie diverse;
- emulatore rimosso; verifica end-to-end su datapath reale con XDP native;
- errore del bias nell'ultimo strato corretto;
- feature dell'interfaccia d'ingresso resa viva in tutte e tre le pipeline;
- **indice del nodo** letto a runtime da tabella in tutte e tre: era l'ultimo ingresso su cui le pipeline non concordavano, e pesa 208 pesi su 319;
- primo strato riscritto con i cicli invertiti in P2 e P3: stesso risultato bit per bit, un terzo dei rami in meno, e il margine di verifica che ne è uscito ha permesso di **recuperare il tetto sulle code** (da 1 a 8) e di **rimettere la riga delle letture di tabella**, che era n.d.;
- oggetto precompilato (AOT) compilato, caricato e misurato: aggiornamento del modello da 1 242 ms a 37 ms, a parità di costo per pacchetto;
- tabella dei risultati rifatta su uno stato del codice solo;
- **analisi parametrica** su cinque assi e tre pipeline: le pendenze sono diverse, ed è la misura che giustifica l'esistenza di tre pipeline invece di una;
- **versione P1 specializzata**: congela anche l'indice del nodo, e toglie alla pipeline hardcoded la dipendenza dalla taglia della rete — al prezzo di un binario per nodo;
- P2 non è più fissa a due hidden layer: la profondità è diventata un involuppo compilato come già lo erano le larghezze — al prezzo, onesto e misurato, di una ricompilazione per profondità.
- **banco end-to-end** con generatore di traffico e nodo sotto prova su core separati (NAPI in modalità thread), condizioni rese identiche fra le pipeline — un fabric solo, tutto compilato prima di misurare, misura a giri — e un **controllo di validità** che il banco esegue su sé stesso: se la baseline non risulta la più veloce, il run viene dichiarato non valido invece che pubblicato;
- **tempo fra arrivo e partenza** misurato per tutte e cinque le pipeline, riproducibile entro pochi nanosecondi, e confermato da due sorgenti di traffico completamente diverse (202 contro 201 ns): è la misura che mancava, ed è quella che dice in nanosecondi quanto costa leggere il descrittore a runtime;
- **dichiarata l'inaffidabilità della portata** su questo banco: ogni riga porta la propria dispersione fra i giri, e sopra il 25% il numero viene marcato invece che riportato come se fosse un risultato.

13.2 Aperto

- **Il tetto del banco è il collegamento virtuale, non IPA.** La veth consegna 1,4–2,1 Mpps con un core in ricezione, e il tentativo di alzarlo cambiando generatore lo ha *abbassato*: il collo di bottiglia è il lato ricezione, non la fabbricazione dei pacchetti. Oltre questa cifra servono due macchine e un cavo, non un'altra manopola.
- **La portata assoluta resta non citabile; la sua differenza no.** A core separati la dispersione fra i giri arriva al 137% e il pavimento è la coda del collegamento virtuale; a core condivisi il tetto è il generatore. In nessuna delle due configurazioni il nodo satura, quindi il numero in Mpps va riportato come limite inferiore. Quello che si cita è il costo per pacchetto *dopo aver sottratto la baseline*: vedi §12.10.
- **Lo sweep sulle taglie di frame** è stato eseguito su 64, 512 e 1514 byte. Il costo dell'inferenza non dipende dalla lunghezza del frame, come atteso; a 1514 byte comincia a pesare la copia dell'headroom. La taglia con la dispersione minore, e quindi quella di riferimento, è **512 byte**.
- **Versione intermedia fra P1 e P2:** esiste ed è stata misurata (hardcoded, pesi nel codice e nodo letto da mappa). Il risultato è negativo in senso utile: su questo banco **non si distingue da P1**. Le due variano fra un run e l'altro più di quanto distino fra loro, e il loro ordine reciproco si inverte. Separarle richiede un banco con meno rumore, non un'altra misura qui.
- **Hardware vero:** la scheda di rete di questa macchina virtuale è emulata e non supporta XDP native. I numeri assoluti restano non citabili come prestazioni di sistema.
- **Il tetto sull'uscita di P2:** lo strato finale è srotolato a 32 classi per un modello che ne ha 7, cioè 256 moltiplicazioni dove ne servono 28. Abbassarlo ridurrebbe molto il programma, ma *cambia il comportamento*: restringe l'insieme dei modelli rappresentabili. La strada pulita è generare quel tetto dall'inviluppo dichiarato dal deployment invece di fissarlo nel sorgente — una ricompilazione per inviluppo, non per modello.
- **I salti fra programmi di P3:** sono tre per pacchetto ed è lì che sta la latenza, non nella dimensione. Ridurli è un lavoro diverso da quello fatto finora.
- **Perché quel modello non si carica:** sappiamo che dipende dai pesi e non dall'architettura, ma non *quale* limite del kernel venga toccato. Il messaggio d'errore ne nomina uno sbagliato. Serve il registro del verificatore, ed è un'ora di lavoro, non una settimana.
- **Rimisurare a macchina scarica:** metà delle latenze dell'analisi parametrica sono state prese mentre la VM era occupata. Le istruzioni no, quelle sono deterministiche.

13.3 Il filo conduttore

Guardando indietro, quasi tutti i problemi trovati a settembre hanno la stessa forma. Non erano calcoli sbagliati: erano **assunzioni non dichiarate** che risultavano vere per caso nell'unica configurazione mai provata.

Assunzione	Vera solo perché
classe k = interfaccia k	le porte erano numerate come le classi
DROP = ultima classe	nel modello depositato capitava così
$n_{\text{nodi}} = 52$ di default	si provava una topologia sola
eth0 = ifindex 2	si provava dentro un container pulito
bias dell'uscita $\times s^0$	si provava coi link tutti attivi
one-hot del nodo = model_id	si registrava un modello solo
tabella vuota = nodo 0	l'array è preallocato a zero
si carica $\Rightarrow$ dipende dall'architettura	non si era mai cambiato solo i pesi

E quasi tutte sono emerse allo stesso modo: cambiando *una* condizione che nessun test aveva mai cambiato.

Un test verde dice che il codice si comporta come il test si aspetta. Non dice che il test stia chiedendo qualcosa.

I comandi

Tutti i comandi si lanciano dalla radice del repository. Le convenzioni sono due: `sudo` serve ovunque si carichi un programma nel kernel o si tocchi un'interfaccia; senza `sudo` girano solo le prove che non escono da Python.

Prerequisiti

Linux con BCC installato. Per il banco di throughput anche il modulo del generatore:

```
sudo modprobe pktgen
```

Su macchina senza BCC le prove che toccano il kernel vengono saltate con un avviso, non falliscono.

14.1 Correttezza: il modello fa quello che deve

Non serve root: girano ovunque, anche su una macchina senza kernel Linux adatto.

```
# tutte le prove disponibili (quelle sul kernel vengono saltate senza BCC)
python3 ipa/test/test_suite.py

# una famiglia alla volta
python3 ipa/test/test_suite.py --only core --verbose
python3 ipa/test/test_suite.py --only quant --samples 200
python3 ipa/test/test_suite.py --only pktstats
python3 ipa/test/test_suite.py --only extract
python3 ipa/test/test_suite.py --only robust

# la catena classe -> azione -> porta logica -> interfaccia
python3 ipa/test/test_class_semantics.py

# topologie sintetiche: il motore non deve dipendere dallo scenario
python3 ipa/test/test_synth.py

# modelli sintetici, le sole vie Python (nessun kernel, nessun root):
# costruzione degli ingressi e confronto float / int8
python3 ipa/test/verify_synth_kernel.py --all --dry-run

# che cosa decide il modello, senza kernel di mezzo
python3 ipa/test/diag_model_decisions.py
```

Le famiglie disponibili per `-only` sono `core`, `pktstats`, `extract`, `quant`, `robust`, `kernel`, `all`.

14.2 Kernel: si carica, e quanto costa

Qui si misura il numero di istruzioni eBPF, la latenza per pacchetto e i salti fra programmi, con `BPF_PROG_TEST_RUN`. È il banco del *tapis roulant*: misura il programma, non la rete.

```
# la tabella delle metriche in-kernel per tutte le pipeline
sudo python3 ipa/test/test_suite.py --only kernel
```

```
# piu' ripetizioni se la macchina e' rumorosa
sudo python3 ipa/test/test_suite.py --only kernel \
    --kernel-repeat 200000 --kernel-trials 15

# piu' modelli caricati insieme: si smistano davvero per model_id?
sudo python3 ipa/test/verify_multi_model.py

# le stesse prove di semantica e topologia, ma dentro al kernel
sudo python3 ipa/test/test_class_semantics.py --kernel
sudo python3 ipa/test/test_synth.py --kernel

# equivalenza NUMERICA sui sette modelli sintetici, nel kernel:
# float / int8 Python / int8 eBPF a confronto, 300 ingressi per scenario.
# La riga che conta e' "int8 (Python) vs int8 (eBPF)": deve dare 100%.
sudo python3 ipa/test/verify_synth_kernel.py --all --n 300
```

14.3 Datapath reale: il pacchetto attraversa davvero

Costruisce una rete di coppie veth, attacca la pipeline in modalità XDP native e verifica che il pacchetto esca dall'interfaccia giusta.

```
# tutte e tre le pipeline sullo stesso fabric
sudo python3 ipa/test/test_fabric.py

# una sola
sudo python3 ipa/test/test_fabric.py --method template

# confronto fra cammino native e cammino generic
sudo python3 ipa/test/test_fabric.py --xdp-mode generic

# se un run si interrompe e lascia interfacce in giro
sudo python3 ipa/test/netns_fabric.py --cleanup
```

14.4 Analisi parametrica: come cresce il costo

Sei assi, tre pipeline, una cella per punto, ognuna in un processo separato perché un programma caricato non contaminerà il successivo.

```
# tutti gli assi
sudo python3 ipa/test/bench_scaling.py

# un asse alla volta: nodes, depth, width, descriptor, isoparam, sparsity
sudo python3 ipa/test/bench_scaling.py --axis nodes
sudo python3 ipa/test/bench_scaling.py --axis depth --trials 15

# salva i CSV (uno per asse)
sudo python3 ipa/test/bench_scaling.py --out results/

# i grafici dai CSV: non serve root, e non rimisura niente
python3 ipa/test/bench_scaling.py --plot results/
python3 ipa/test/bench_scaling.py --plot results/ --all-plots

# le pipeline danno tutte la stessa risposta? (80 confronti)
sudo python3 ipa/test/bench_scaling.py --verify

# larga contro profonda, a parità di parametri
sudo python3 ipa/test/bench_depth_vs_width.py
```

```
sudo python3 ipa/test/bench_depth_vs_width.py --descriptor no_onehot
```

14.5 Throughput e latenza end-to-end

Il banco con generatore e nodo sotto prova. **La forma da citare è quella con -latency**: un fabric solo, tutto compilato prima di misurare, e misura a giri con la mediana fra i giri.

```
sudo modprobe pktgen

# il confronto equo: latenza arrivo->partenza + portata, tutte le pipeline
sudo python3 ipa/test/bench_throughput.py --latency --rounds 5 --out result/

# piu' taglie di frame
sudo python3 ipa/test/bench_throughput.py --latency \
    --frames 64,512,1514 --rounds 5 --out result/

# piu' giri se la dispersione fra i giri sfonda il 25%
sudo python3 ipa/test/bench_throughput.py --latency --rounds 7 --out result/

# se un run si interrompe e lascia il generatore configurato
sudo python3 ipa/test/bench_throughput.py --cleanup
```

Scrivi due file: `latency.csv` con la mediana fra i giri e `latency_raw.csv` con ogni singolo giro. Il secondo serve a leggere la dispersione, che è ciò che dice se la mediana significa qualcosa.

Come si legge l'uscita

Due verdetti, in fondo al run, e vanno letti separatamente:

Controllo di validità — la baseline deve essere la più veloce entro il 10%. Se non lo è, il run misura il carico della macchina e va buttato.

Latenza: quanto è riproducibile — se il minimo varia meno del 5% fra i giri, è la cifra da citare. Vale anche quando la portata è inutilizzabile: sono due misure con due affidabilità diverse.

14.6 Costi specifici

```
# quanto costa un salto fra programmi
sudo python3 ipa/test/bench_tailcall_overhead.py
sudo python3 ipa/test/bench_tailcall_overhead.py --repeat 5000 --trials 15

# quanto costa aggiungere un modello a caldo
sudo python3 ipa/test/bench_model_add.py
sudo python3 ipa/test/bench_model_add.py --n-models 3
```

14.7 Oggetto precompilato (AOT)

Genera il sorgente C dai pesi, lo compila e lo carica con un loader in C, senza Python né compilatore sul nodo.

```
make -C ipa/poc_aot                # genera, compila, linka
make -C ipa/poc_aot run            # compila e carica

# a mano, scegliendo il nodo e l'interfaccia
```

```
cd ipa/poc_aot
sudo ./loader_aot nn_aot_arch.o --node-id 7 --attach <ifindex>
sudo ./loader_aot nn_aot_arch.o --xdp-mode generic

make -C ipa/poc_aot clean
```

14.8 Diagnostica, quando qualcosa non torna

```
# due pipeline decidono diversamente: su quale ingresso?
sudo python3 ipa/test/diag_fabric_divergence.py
sudo python3 ipa/test/diag_fabric_divergence.py --link-state 100001 --ttl 28

# da quale commit P3 ha smesso di caricarsi
sudo python3 ipa/test/diag_p3_bisect.py --rev <commit> --rev HEAD
```

14.9 L'ordine consigliato

Partendo da zero su una macchina nuova:

1. `python3 ipa/test/test_suite.py` — il modello è sano?
2. `sudo python3 ipa/test/test_suite.py -only kernel` — si carica, e quanto costa?
3. `sudo python3 ipa/test/test_fabric.py` — il pacchetto attraversa davvero?
4. `sudo python3 ipa/test/bench_scaling.py -verify` — le tre pipeline concordano?
5. `sudo python3 ipa/test/bench_throughput.py -latency`
— quanto tempo passa fra arrivo e partenza?

I primi quattro rispondono *sì* o *no*. Solo il quinto produce un numero che va interpretato, ed è anche l'unico che può fallire per colpa della macchina invece che del codice.